University of Glasgow | School of Computing Science

Honours Individual Project Dissertation

TriBench: A Framework for Benchmarking SQL Workloads on Distributed Data Lakehouses

2736414Y

March 26, 2026

Abstract

Benchmarking distributed SQL systems is significantly harder than benchmarking traditional databases: rather than a single system, a modern data lakehouse deployment relies on several coordinated services that must all be configured, started, and monitored together. Existing tools for the Apache Trino query engine either assume the cluster is already running, lack hardware monitoring, or embed environment-specific assumptions that prevent experiments from being reproduced on different machines.

TriBench addresses this gap as a cross-platform benchmarking framework for distributed data lakehouses. Experiments are defined declaratively and run unchanged across a local single-machine deployment or a multi-node cloud cluster by switching a single configuration file. Hardware and resource metrics are collected automatically alongside query execution, and all results are packaged into a self-contained, portable bundle that can be shared and reproduced without additional setup.

TriBench was evaluated across three environments of increasing scale and complexity: a local Docker deployment, a self-managed on-premises Kubernetes cluster, and Google Kubernetes Engine. Using industry-standard SQL benchmark suites at dataset sizes up to 100 GB, the framework produced reproducible query timings with full telemetry in all three environments, demonstrating the end-to-end deployment, orchestration, and benchmarking capability that the Trino lakehouse ecosystem has lacked.

Acknowledgements

I would like to thank my supervisor, Professor Lauritz Thamsen, for his guidance and support throughout this project, and for the insightful weekly meetings that helped shape both the direction and rigour of this work.

I would also like to thank my friends for making my time at university so memorable, in particular Suajy Patel, Oran Grant, Dylan Huang, and Ravin Aziz for taking the time to review this dissertation and for their thoughtful feedback and discussion.

Finally, I would like to thank my family for their unwavering support and encouragement throughout my studies. I am grateful to close this chapter of my life with so many fond memories, and I look forward with excitement to whatever comes next.

Goodbye, Glasgow!

Education Use Consent

I hereby grant my permission for this project to be stored, distributed and shown to other University of Glasgow students and staff for educational purposes. **Please note that you are under no obligation to sign this declaration, but doing so would help future students.**

Signature: Lin Yuan Date: 27 March 2026

Contents

1 Introduction **1**
- 1.1 Motivation 1
- 1.2 Aims 1
- 1.3 Outline 2

2 Background **3**
- 2.1 Data Lakehouse Architecture 3
- 2.2 Benchmarking for Distributed SQL Systems 4
- 2.3 TPC Benchmark Standards 4
- 2.4 Reproducibility in Systems Research 5

3 Related Work **6**
- 3.1 Benchmarking Frameworks for Distributed Systems 6
  - 3.1.1 PEEL 6
  - 3.1.2 LST-Bench 7
  - 3.1.3 Benchto 7
- 3.2 Lakehouse Technologies and Container Orchestration 7
  - 3.2.1 Apache Iceberg 7
  - 3.2.2 Apache Trino 8
  - 3.2.3 Kubernetes 9

4 Analysis and Requirements **10**
- 4.1 User Scenarios 10
- 4.2 Requirements Set 10
  - 4.2.1 Functional Requirements 10
  - 4.2.2 Non-Functional Requirements 11
- 4.3 Summary 11

5 Design **12**
- 5.1 Architectural Overview 12
- 5.2 Open Lakehouse Stack 13
- 5.3 CLI Layer 13
- 5.4 Bundle Abstraction 14
- 5.5 Configuration Layer 14
- 5.6 Orchestration Layer 15
  - 5.6.1 Experiment Definitions 15
  - 5.6.2 Execution Lifecycle 16

5.7 Monitoring System 17
5.8 Systems Layer 17
5.9 Dataset Generation and Loading 18
5.10 Result Storage 18
5.11 Summary 19

6 Implementation 20
6.1 Development Approach 20
6.1.1 Language and Tooling 20
6.1.2 Version Control and Testing 20
6.2 CLI Implementation 20
6.3 Bundle Implementation 21
6.4 Configuration Loading 21
6.5 Dataset Loading 22
6.5.1 Dataset Generation 22
6.5.2 Universal Loader 23
6.6 Experiment Execution 24
6.6.1 Single Experiment 24
6.6.2 Suite Execution 24
6.6.3 Parallel Query Execution 24
6.7 Concurrent Monitoring 24
6.7.1 Threading Model 24
6.7.2 Host Resource Collector 25
6.7.3 Kubernetes Pod Collector 25
6.8 Deployment Backends 26
6.8.1 Docker Backend 26
6.8.2 Kubernetes Backend 26
6.9 Result Storage 26
6.9.1 Persistence 26
6.9.2 Export 26
6.9.3 Built-in Analysis 27
6.10 Extending TriBench 27
6.11 Summary 27

7 Experiments 28
7.1 Unit Testing 28
7.2 Experiment Methodology 28
7.3 Experimental Setup 28
7.3.1 General Experiment Workflow 28
7.3.2 Local Docker Environment 29
7.3.3 On-Premises Kubernetes Environment 29
7.3.4 GKE Cloud Environment 29
7.4 Local Docker Deployment 29
7.4.1 Deployment and Configuration 30
7.4.2 Experiment Execution 30

7.4.3 Host Monitoring 30
7.4.4 Result Storage and Export 31
7.5 On-Premises Kubernetes Deployment 31
7.5.1 Kubernetes Deployment 31
7.5.2 TPC-DS Workload 32
7.5.3 Custom Dataset 32
7.6 GKE Cloud Deployment 33
7.6.1 GKE Deployment 33
7.6.2 Worker Scaling Study 33
7.6.3 Full Suite at Four Workers 34
7.6.4 TPC-H SF100 Query Performance 35
7.7 Cross-Environment Portability 36
7.8 Framework Overhead 37
7.9 Reproducibility 38
7.10 Evaluation Limitations 38
7.11 Summary 38

8 Conclusion 39
8.1 Summary 39
8.2 Reflection 39
8.3 Future Work 40

Appendices 41

A CLI Command Reference 41

B Getting Started Guide 43

C Requirements Justification 46

D Screenshots 47

E Source Code Directory Structure 57

Bibliography 59

1 Introduction

This chapter outlines the motivation for developing TriBench, a cross-platform benchmarking framework for SQL workloads on distributed data lakehouses. The key challenges in the problem space are identified, and the overall aims of the project are introduced.

1.1 Motivation

The data lakehouse has emerged as the dominant pattern for large-scale analytical data management, combining the low-cost storage of a data lake with the ACID guarantees and query performance of a data warehouse (Zaharia et al. 2021). Apache Trino (Sethi et al. 2019), a distributed SQL query engine originally developed at Meta, is widely used as the interactive query layer over such systems, with Apache Iceberg (Apache Software Foundation 2026*b*) providing the underlying open table format.

Benchmarking such a stack is considerably harder than benchmarking a traditional RDBMS. Rather than a single system under test, a Trino lakehouse deployment relies on several coordinated services that must all be configured, started, and stopped together. Each service carries hardware-dependent settings that must be tuned consistently before any experiment can run, and these settings must change when the deployment environment changes. Studying how the system scales, for instance by increasing the number of query processing nodes, requires changes across several components and a full redeployment of the stack. When this is done manually, the exact configuration used is rarely captured alongside the results, making it difficult for others to reproduce the same experiment on different hardware. An experiment tuned for a laptop may produce out-of-memory errors or skewed timings on a cluster node, with nothing in the results to indicate that the configuration was the cause.

This challenge is not unique to Trino. The Apache Spark ecosystem faced the same problem, and the PEEL framework (Boden et al. 2017) was developed to address it, automating the setup, execution, and teardown of multi-system experiments and packaging them into portable, reproducible units. For Trino, Benchto (PrestoDB 2026) covers query execution and result storage but does not address the full deployment lifecycle. Without a tool that handles the full stack, practitioners typically resort to ad-hoc scripts that embed machine-specific assumptions and store results as unstructured logs, making systematic comparison and reproduction difficult. TriBench is designed to fill this gap: a framework for defining, running, and reproducing SQL benchmark experiments across Trino deployments, from a local machine to a cloud cluster.

1.2 Aims

TriBench targets engineers and researchers who need to evaluate SQL query performance on distributed data lakehouses in a systematic and reproducible way. The framework is designed around five goals. First, it must run SQL benchmark workloads against a Trino cluster from a self-contained experiment definition, supporting both industry-standard suites and user-supplied queries. Second, it must collect hardware and system resource metrics concurrently with query

execution, capturing data at the host level and, on distributed cluster deployments, at the individual service level. Third, results must be stored in a structured form that supports post-hoc analysis and comparison across runs. Fourth, the same experiment definition must run unchanged across deployment environments by changing only a deployment configuration, covering both a local single-node setup and a multi-node cloud cluster. Fifth, all experiment artefacts must be packaged into a portable, self-contained unit that can be archived and reproduced on a different machine.

1.3 Outline

- **Chapter 2** introduces the foundational concepts: the data lakehouse architecture, Apache Iceberg, Apache Trino, and the principles of reproducible benchmarking.
- **Chapter 3** surveys related work, examining existing benchmarking frameworks, including PEEL for Apache Spark and Benchto for Trino, and identifying the gap that TriBench addresses.
- **Chapter 4** presents a structured set of functional and non-functional requirements derived from the aims above.
- **Chapter 5** describes the design of TriBench: its five-layer architecture, configuration system, monitoring subsystem, and result storage schema.
- **Chapter 6** explains how the design was realised in Python, covering key libraries, the execution pipeline, and implementation challenges.
- **Chapter 7** evaluates TriBench across three deployment environments, assessing requirement coverage, performance, and reproducibility.
- **Chapter 8** concludes with a summary, critical reflection, limitations, and directions for future work.

2 Background

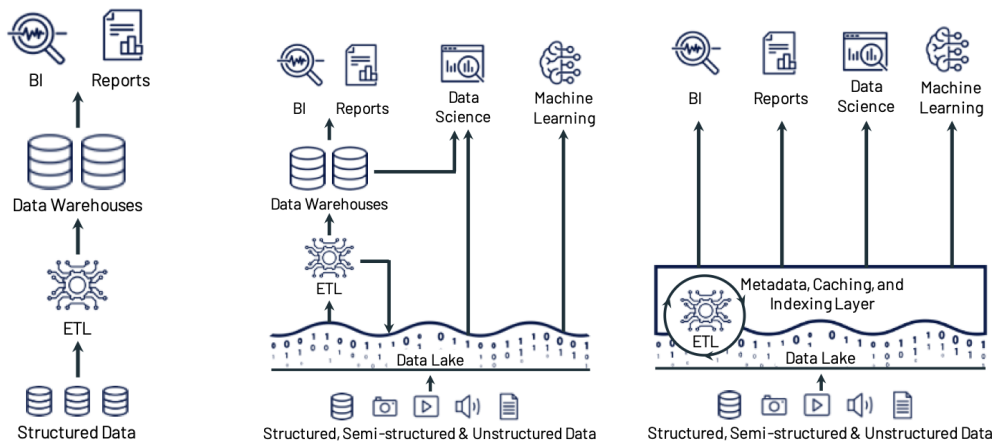
2.1 Data Lakehouse Architecture

The evolution of modern analytics platforms can be understood as a progression from data warehouses to data lakes, and finally to the data lakehouse pattern (Zaharia et al. 2021), as summarised in Figure 2.1. Early data warehouses centralised curated data for decision support and business intelligence (Kimball and Ross 2013). They typically used a schema-on-write approach: data was transformed on ingest into a fixed model designed for predictable analytical queries. This provided strong governance and excellent performance, but often at the cost of tight coupling between storage and compute, expensive scaling as data volumes grew, and limited support for semi-structured or unstructured data.

Data lakes emerged to address these constraints by separating storage from compute. Raw data could be stored cheaply in generic file systems or object stores, commonly using open columnar formats (Abadi et al. 2008; Abadi 2012) such as Apache Parquet (Apache Software Foundation 2026*c*). This schema-on-read model improved agility and reduced cost, but it introduced new problems: without a transactional coordination layer and consistent metadata, concurrent writers can leave a dataset in an inconsistent state, and query engines may be forced into large scans because they lack reliable file- and partition-level statistics.

The *data lakehouse* approach aims to combine the openness and cost profile of data lakes with the management properties of warehouses. It does so by introducing an open table format layer that maintains transaction state and query-relevant metadata alongside the data files. Apache Iceberg (Apache Software Foundation 2026*b*) is a prominent implementation. An Iceberg table is defined by a metadata tree: a *metadata file* points to the current *snapshot*, which references a *manifest list* of *manifest files* describing the underlying Parquet objects, including column statistics. This metadata enables optimisations such as partition pruning and predicate pushdown, substantially reducing the amount of data that must be read for many analytical queries.

To make these tables discoverable via SQL, Iceberg deployments use a catalogue service that maps logical table names to metadata locations. In TriBench's stack, the Hive Metastore Service (Thusoo et al. 2009) fulfils this role, persisting catalogue state in a relational backend. Apache Trino (Sethi et al. 2019), the distributed SQL engine that forms the compute layer of the stack, consults the catalogue during query planning to resolve the current metadata pointer for each referenced table. Trino operates under a coordinator–worker model: a coordinator node parses and plans queries, then distributes execution as parallel tasks across worker nodes that read Parquet data directly from object storage. This architecture decouples query coordination from data processing, meaning the cluster can scale horizontally by adding worker nodes without any change to the queries being executed.



(a) Data Warehouse *(b) Data Lake* *(c) Data Lakehouse*

Figure 2.1: *Evolution of data platform architectures to today's two-tier model (a-b) and the new Lakehouse model (c). Zaharia et al. (2021)*

2.2 Benchmarking for Distributed SQL Systems

According to Gray (1993) and Huppler (2009), a useful benchmark must possess four properties: relevance, portability, scalability, and simplicity. The Transaction Processing Performance Council (TPC) provides the industry-standard suites that satisfy these criteria for analytical SQL workloads; the specific benchmarks used in this work are described in Section 2.3.

Benchmarking distributed SQL engines introduces significant complexity beyond single-node systems. Because queries execute as parallel jobs across multiple machines, end-to-end latency reflects not just computational work but also scheduling overhead, network exchanges, and remote storage throughput. This makes reproducibility (discussed in Section 2.4) a central concern: without tight control over environment, data layout, and execution parameters, results from one run cannot be meaningfully compared with another.

2.3 TPC Benchmark Standards

Founded in 1988, the Transaction Processing Performance Council (TPC) is a non-profit organisation dedicated to developing and maintaining data-centric benchmark standards and disseminating objective, verifiable performance data to industry (Transaction Processing Performance Council 2026*b*). TPC benchmarks are divided into *Enterprise* benchmarks (technology-agnostic, specification-based standards where vendors choose their own implementation) and *Express* benchmarks, which are kit-based with shorter development cycles.

TPC-H (Transaction Processing Performance Council 2026*b*) is a decision-support benchmark that models a supply-chain scenario with 22 business-oriented ad-hoc queries. The dataset and queries are designed for broad industry relevance, exercising decision-support systems that examine large volumes of data and execute queries with a high degree of complexity. Scale is controlled by a scale factor (SF), where SF 1 generates approximately 1 GB of raw data, SF 10 generates 10 GB, and so on, across eight related tables. TPC-H is the primary built-in benchmark suite shipped with TriBench: all 22 queries are included, pre-adapted to Trino-compatible SQL and parameterised by Iceberg catalogue and schema name so they can be used out of the box against a loaded dataset.

TPC-DS (Nambiar and Poess 2006; Poess et al. 2017; Transaction Processing Performance Council 2026*a*) extends the decision-support model to a complex retail scenario with 99 queries testing advanced SQL features such as window functions, complex joins, and nested subqueries. TPC-DS is the second built-in benchmark suite: its query files are similarly stored in TriBench's directory, and TriBench's dataset registry and query-loader are structured to load and execute them against an Iceberg schema in exactly the same way as TPC-H, without any framework modification. Both suites form the primary workloads in the evaluation (Chapter 7).

2.4 Reproducibility in Systems Research

Jain (1990) defines reproducibility as the ability for an independent experimenter to obtain consistent results under identical conditions. In systems research, meeting this definition in practice requires capturing not just the software configuration but also the hardware environment, dataset state, and execution parameters, all of which interact in a distributed SQL system.

The distributed systems research community has long recognised this challenge (Collberg and Proebsting 2016). A typical performance study requires coordinating multiple interdependent services (storage, catalogue, and compute engine), each with its own configuration. Without a systematic means of recording and replaying these configurations, reported results are difficult to verify and cannot be reproduced on a different machine. Boden et al. (2017) articulated this directly: practitioners are typically forced to choose between manually steering experiment lifecycles, an error-prone and non-reproducible process, or writing ad-hoc scripts that embed environment-specific assumptions. Both approaches produce artefacts that cannot be shared or re-run without significant manual effort.

Achieving reproducibility in distributed SQL benchmarking requires strict control over four dimensions that directly motivate the design of TriBench:

1. **Environment Specification:** Precise control over cluster configuration, including worker counts and memory limits, and JVM heap settings.
2. **Data Consistency:** Loading the dataset at an identical scale factor with the same physical layout and Iceberg partitioning strategy across runs.
3. **Execution Control:** Standardising the warmup policy, query execution order, and concurrency model so that variance between runs reflects only system behaviour.
4. **Measurement Capture:** Recording both query-level statistics (wall-clock time, CPU time, bytes scanned) and concurrent system-level telemetry (CPU utilisation, memory pressure). Capturing both is essential, as resource saturation often manifests in hardware metrics before it is visible in query latency.

The TriBench design maps each of these dimensions to a specific mechanism, as described in Chapter 7.

3 Related Work

This chapter surveys benchmarking frameworks and deployment technologies informing TriBench's design. Section 3.1 reviews specialised frameworks for distributed data processing: PEEL, LST-Bench, and Benchto. Section 3.2 describes the Trino–Iceberg lakehouse stack and Kubernetes container orchestration.

3.1 Benchmarking Frameworks for Distributed Systems

3.1.1 PEEL

PEEL (Boden et al. 2017; 2015) is a framework developed at TU Berlin for reproducible experiments on distributed data processing systems such as Apache Spark (Zaharia et al. 2016), Flink (Carbone et al. 2015), and Hadoop (Shvachko et al. 2010). It structures the benchmarking process into a continuous four-stage life cycle: Define, Execute, Analyse, and Share, as illustrated in Figure 3.1.

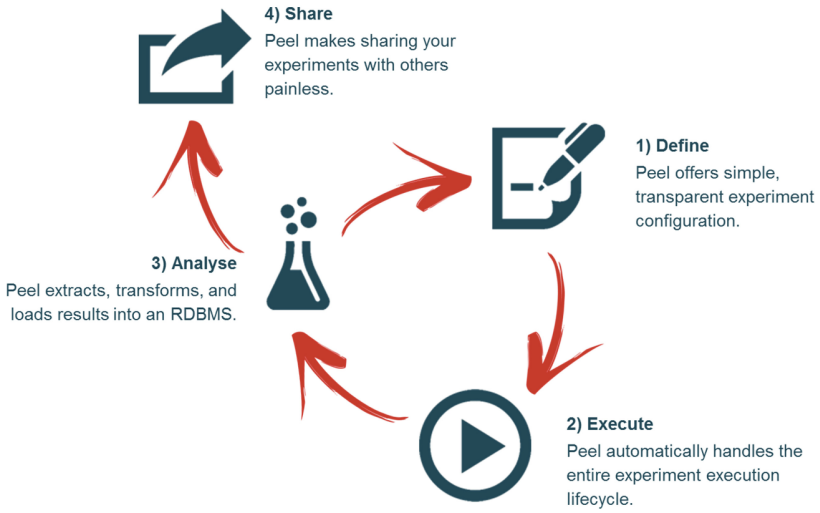


Figure 3.1: *The four stages of the PEEL experiment life cycle. Figure from Boden et al. (2015).*

In the **Define** stage, experiments are configured through a layered HOCON (Lightbend 2026) hierarchy that separates framework defaults, bundle-level settings, and host-specific overrides. This allows the same experiment to run on different hardware without modification, a design pattern TriBench applies to the Trino lakehouse stack. In the **Execute** stage, PEEL automatically starts and stops each service based on a declared lifecycle scope, and triggers hardware telemetry collection alongside execution. In the **Analyse** stage, results are extracted from log files into a relational database, which feeds into the **Share** stage. The unit of sharing is the *bundle*: a

self-contained, portable directory packaging workload applications, configuration files, datasets, and result logs. Bundles can be pushed to remote clusters and shared with collaborators, who can reproduce the full experiment without additional setup.

PEEL's bundle paradigm and layered configuration model are proven design patterns, but two fundamental gaps prevent their application to modern lakehouse stacks. PEEL targets Hadoop-era engines on bare-metal or YARN, with no support for Kubernetes or the Trino–Iceberg open lakehouse composition. Its telemetry is post-hoc, collected from log files after runs complete. TriBench applies these proven design patterns to solve a different problem: end-to-end lifecycle management of a complete open lakehouse stack across local and distributed deployments, with concurrent hardware telemetry and per-query statistics captured directly from the query engine during execution. No existing framework provides this full lifecycle capability for Apache Trino.

3.1.2 LST-Bench

LST-Bench (Camacho-Rodríguez et al. 2024) evaluates Log-Structured Tables (Delta Lake (Armbrust et al. 2020), Apache Iceberg, Apache Hudi (Apache Software Foundation 2026*a*)) on cloud object storage. Developed by Microsoft Research, it extends TPC-DS with composable workload patterns addressing LST-specific characteristics: longevity, resilience, read/write concurrency, and time travel. It quantifies degradation rate as delta files accumulate and automates telemetry from compute engines and cloud storage APIs to measure latency, throughput, storage efficiency, and compute utilization. LST-Bench compares table format performance and assumes infrastructure is already provisioned; it is a workload tool rather than a deployment harness. Where LST-Bench evaluates how a table format behaves under load, TriBench manages the environment that workloads run on.

3.1.3 Benchto

Benchto (PrestoDB 2026) is a macro-benchmarking framework for distributed SQL engines, maintained under the official Trino GitHub organisation. It is the closest existing tool to TriBench in purpose: benchmark definitions are declared in YAML, a driver process submits queries to the target engine, and results are persisted in a PostgreSQL-backed service for comparison across runs.

Benchto addresses query execution and result storage well, but it assumes the cluster is already provisioned and running. It provides no mechanism to set up or tear down the underlying lakehouse services, and no support for switching between local and distributed deployment environments. There is no data loading lifecycle, no bundle abstraction for packaging experiments, and no concurrent hardware telemetry. TriBench fills these gaps: it manages the full lakehouse service lifecycle, collects hardware and resource metrics concurrently with query execution, and packages all artefacts into a portable bundle that can be reproduced without additional setup.

3.2 Lakehouse Technologies and Container Orchestration

This section surveys the core technologies that form the open lakehouse stack evaluated in this work, along with the container orchestration platform that enables reproducible distributed deployments.

3.2.1 Apache Iceberg

Apache Iceberg is an open table format that manages how data files are organised, versioned, and exposed as a single queryable table (Apache Software Foundation 2026*b*). To achieve this, Iceberg separates concerns into distinct catalog, metadata, and data layers, as illustrated in Figure 3.2. At

the top, the Iceberg Catalog maintains a pointer to the current metadata file. This metadata layer records the schema, partitions, and per-file statistics. It branches down into manifest lists and individual manifest files, which ultimately track the actual data files residing in the data layer. Query engines traverse this hierarchical metadata to safely skip irrelevant files before reading the underlying data, drastically reducing unnecessary I/O.

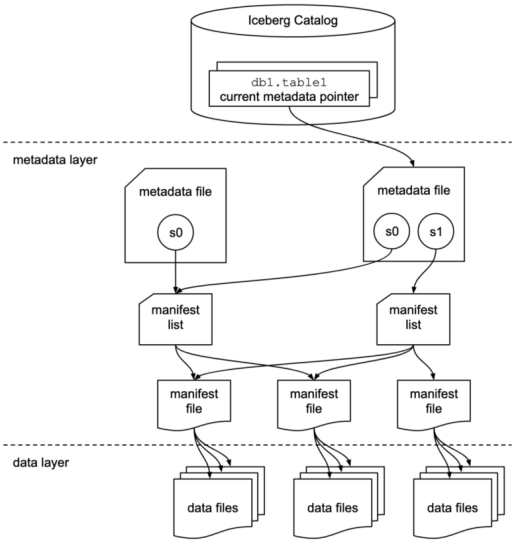


Figure 3.2: *Iceberg table format. Figure from Daniel Palma (2022).*

This layered architecture natively supports snapshot-based versioning, represented by the s0 and s1 states in the diagram's metadata files. Each write produces a new snapshot and atomically updates the catalog's metadata pointer, ensuring that readers always see a consistent view even during concurrent writes. For TriBench, Iceberg ensures that TPC-H and custom datasets are loaded in a consistent, versioned state, meaning benchmark results purely reflect query engine behaviour rather than underlying data inconsistencies.

3.2.2 Apache Trino

Apache Trino (formerly PrestoSQL) is a distributed SQL query engine (Sethi et al. 2019; Fuller et al. 2021) built on a strictly separated coordinator–worker architecture, as detailed in Figure 3.3. The lifecycle of a query begins when a Client submits a statement to the Coordinator. Inside the Coordinator, the query passes through a Parser & Analyzer, a Planner, and a Scheduler, which jointly translate the SQL into an optimised, parallelisable execution plan. The Scheduler then distributes these tasks to multiple Worker nodes. The workers perform the heavy lifting: they execute the plan, read data directly from the underlying storage, and stream results back. Because the architecture decouples query coordination from data processing, scaling the cluster simply means adding more workers; the query and experiment definitions stay unchanged.

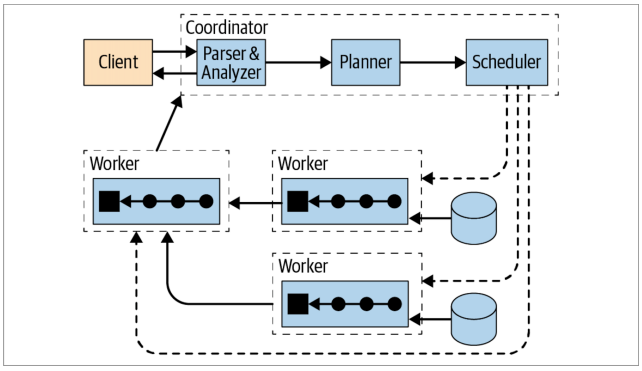


Figure 3.3: *Trino architecture overview with coordinators and workers. Figure from Trino Project (2021).*

As the workers execute these scheduled tasks, CPU, memory, disk I/O, and network resources heavily dictate Trino's overall performance. Workers consume CPU through sorting, hashing, and aggregation. Large joins and aggregations can exhaust a worker's available memory, causing disk spillage and sharply increasing query time. Furthermore, network throughput becomes a primary bottleneck when workers must exchange intermediate results between execution stages (indicated by the data flow arrows between workers in Figure 3.3). These complex, distributed interactions make concurrent hardware monitoring essential, as a slowdown may originate in any of these interconnected resources.

3.2.3 Kubernetes

Kubernetes (The Kubernetes Authors 2026) is the container orchestration platform enabling reproducible distributed deployments in this work. Where a single-machine deployment runs all services on one host, Kubernetes distributes them across cluster nodes, providing stable network identities, persistent storage, and service discovery so that components locate and connect regardless of which node they are scheduled on. This abstraction is particularly valuable for stateful data systems such as query engines and catalogue services, which depend on reliable inter-service communication and persistent storage across restarts.

Kubernetes also enables scaling studies by allowing the number of compute nodes to be adjusted through configuration alone, without modifying the workloads being benchmarked. The same deployment specification runs unmodified on a local single-node cluster or a multi-node production environment, making it well suited to reproducible distributed benchmarking. Chapters 5 and 6 discuss how TriBench exploits these properties across local and cloud deployments.

4 Analysis and Requirements

This chapter derives a prioritised set of functional and non-functional requirements from user scenarios. Requirements were elicited through supervisor discussions, a prototyping phase, and lessons from PEEL, LST-Bench, and Benchto (Chapter 3).

4.1 User Scenarios

Three scenarios cover distinct deployment contexts, each demonstrating a different aspect of TriBench's flexibility across environments.

Scenario 1: Local baseline. A researcher wants to evaluate SQL query performance on a data lakehouse but has no prior experience configuring the underlying services. They create a self-contained experiment bundle, activate a local deployment profile, and run a standard benchmark workload. TriBench provisions the full stack automatically, performs a configurable number of warmup and measured runs per query with host resource monitoring, then tears down the environment. Results are exported to a common format for inspection.

Scenario 2: On-premises cluster study. A researcher benchmarks a distributed query engine on a self-managed on-premises cluster. They switch to a cluster deployment profile and run both a standard benchmark suite and a custom dataset, with one warmup and five measured repetitions. TriBench collects per-service CPU and memory usage alongside host-level telemetry, with no changes to the experiment definition from Scenario 1.

Scenario 3: Cloud scaling study and sharing. An engineer runs a scaling study on a managed cloud cluster, executing a standard benchmark workload at increasing node counts to observe how query execution time changes with cluster size. They then run a full benchmark suite across multiple dataset sizes unattended overnight, with the cluster fixed at maximum capacity. The complete experiment; including all configurations, datasets, and results is archived into a portable bundle and shared with a colleague, who reproduces the full experiment without any additional setup.

4.2 Requirements Set

Requirements are prioritised using MoSCoW (Clegg and Barker 1994): **M** must-have, **S** should-have, **C** could-have, **W** won't have.

4.2.1 Functional Requirements

Workload Execution (Goal 1). *Execute SQL workloads against Trino from a declarative definition, supporting standard suites and custom queries.*

1. **M1 – Declarative execution:** Experiments are orchestratable via YAML files without modifying framework code.
2. **M2 – Benchmark workloads:** TPC-H and TPC-DS are supplied; custom queries and user datasets are fully supported.

3. **M3 – Execution control:** Warmup runs, measured runs, timeouts, and validation thresholds are configurable.
4. **S1 – Suite abstraction:** Multiple experiments can be grouped into suites executable as a single CLI operation.

Hardware Monitoring (Goal 2). *Sample host CPU, memory, and I/O concurrently with query execution, alongside per-pod Kubernetes telemetry and query engine statistics.*

1. **M4 – Telemetry collection:** Host-level resources are sampled concurrently while per-query engine statistics are retrieved after each execution.
2. **S2 – Cloud metrics:** Kubernetes pod metrics are collected and normalised alongside host telemetry.

Result Storage (Goal 3). *Persist results and telemetry non-destructively, supporting cross-run analysis and export to common formats.*

1. **M5 – Structured storage:** Results and telemetry are stored non-destructively in a relational schema.
2. **M6 – Data export:** Stored runs can be exported to CSV, JSON, and Parquet.
3. **M7 – Built-in analysis:** Basic statistical evaluation including performance, query comparison, and monitoring summaries is provided.

Deployment Backends (Goal 4). *Run the same experiment unchanged across Docker and Kubernetes by switching only a host configuration profile.*

1. **M8 – Lifecycle abstraction:** Setup, execution, and teardown are automated across interchangeable Docker and Kubernetes backends.
2. **M9 – Configuration profiles:** Users switch deployment environments without modifying experiment definitions.
3. **W1 – Flexible stack:** Supporting alternative SQL engines is out of scope.

Bundle Abstraction (Goal 5). *Package all experiment artefacts into a self-contained bundle that can be archived and reproduced on any machine.*

1. **M10 – Self-contained bundles:** Benchmarks are packaged as shareable bundles containing all configurations and datasets.
2. **M11 – Bundle management:** The CLI scaffolds and activates bundles to direct all subsequent operations.

4.2.2 Non-Functional Requirements

1. **M12 – Reproducibility:** Identical experiments on identical hardware produce consistent results.
2. **M13 – Portability:** The framework executes unmodified across macOS and Linux.
3. **M14 – Observability:** Structured timestamped logs are emitted at configurable verbosity levels.
4. **S3 – Low overhead:** Background monitoring perturbs query execution times by less than 5% by default.
5. **C1 – Extensibility:** New deployment backends can be integrated via an abstract system interface.

4.3 Summary

Three user scenarios motivated 14 must-have, three should-have, one could-have, and one won't-have requirement. Chapter 5 presents the architecture satisfying them; a full traceability table is provided in Appendix C.

5 Design

TriBench is organised as a five-layer architecture with two cross-cutting concerns. Each layer satisfies a distinct subset of requirements from Chapter 4; the design decisions around the configuration model, backend abstraction, and concurrent monitoring are motivated below alongside each component.

5.1 Architectural Overview

TriBench's five layers are shown in Figure 5.1: CLI, configuration, orchestration, systems, and data lakehouse. Control flows downward: a CLI command triggers configuration resolution, which parameterises orchestration, which drives system lifecycle management, which starts and stops the lakehouse services. Telemetry flows upward via two cross-cutting concerns: monitoring spans layers three to five, and result storage spans layers three to four.

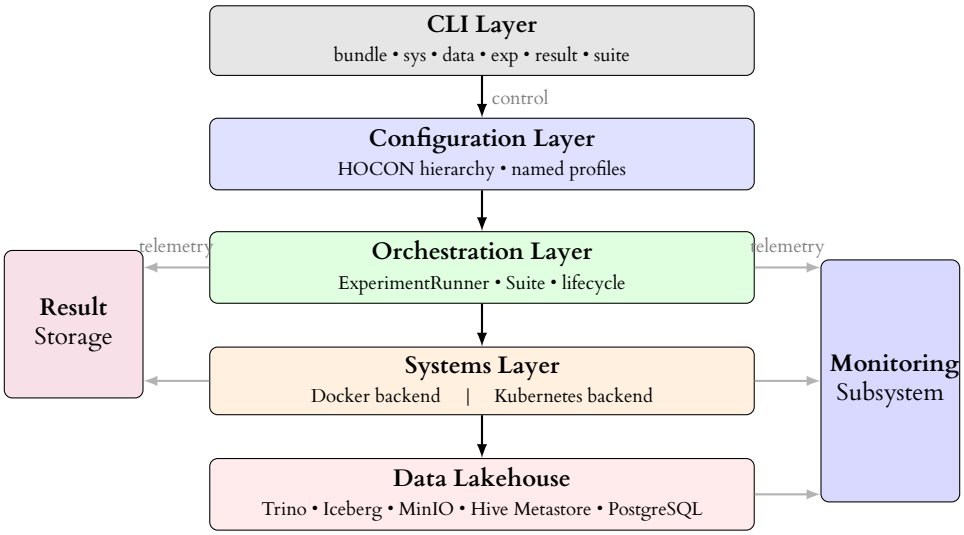


Figure 5.1: *Five-layer architecture. Control flows downward through the layers; monitoring and result storage span layers three to five and three to four respectively as cross-cutting subsystems.*

The layering addresses three non-functional requirements. **M13** (platform portability) is met by confining all platform-specific logic to the systems layer, so nothing above it contains environment assumptions. **C1** (extensibility) is a consequence: new deployment backends require only a concrete implementation of the System abstract base class in the systems layer, with no changes elsewhere. Two cross-cutting subsystems span the orchestration and systems layers: monitoring (Section 5.7) collects hardware telemetry concurrently with query execution (**M4**, **S2**), and result storage (Section 5.10) persists run records to a structured database inside the bundle (**M5**, **M6**).

5.2 Open Lakehouse Stack

The framework evaluates a complete open lakehouse stack. **Trino** (Section 3.2.2) serves as the primary distributed SQL engine, chosen because unlike Spark and Flink, it previously lacked a dedicated benchmarking harness like PEEL. The stack is completed by three supporting components: **Apache Iceberg** (Section 3.2.1) manages the open table format and versioned data storage; an S3-compatible object store (**MinIO**, (MinIO Inc. 2026)) holds the underlying data files; and a catalogue service (**Hive Metastore** (Thusoo et al. 2009), backed by **PostgreSQL** (The PostgreSQL Global Development Group 2026)) maintains the table metadata that query planning depends on. Depending on the scale of the experiment, this entire stack can be deployed locally on a single machine or distributed across a cluster, without any change to the experiment definitions. The default component versions are Trino 434, Hive Metastore 4.0.0, PostgreSQL 15, and MinIO RELEASE.2025-10-15; alternative versions can be specified via the configuration layer, though compatibility is not guaranteed as API contracts and configuration interfaces may differ across releases. Figure 5.2 shows how the four services compose.

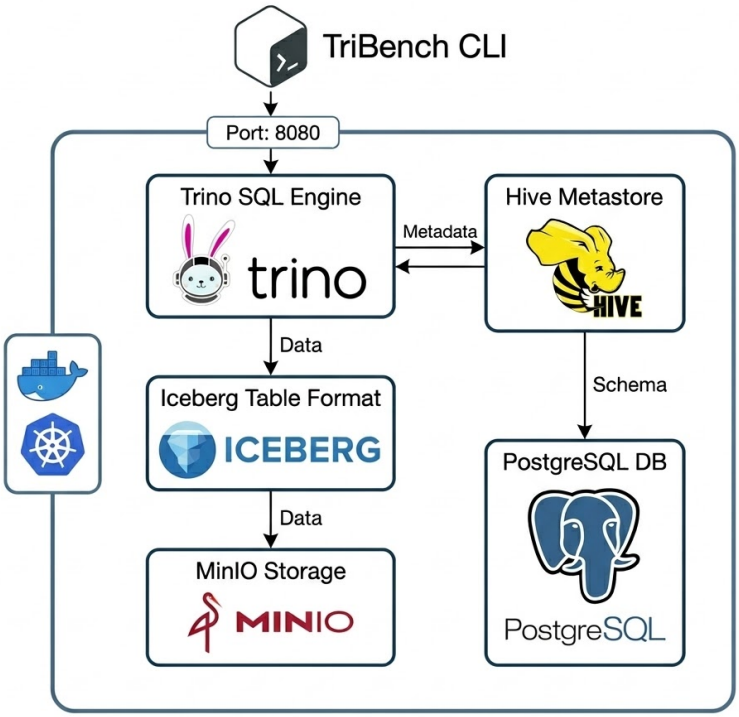


Figure 5.2: *TriBench open lakehouse stack. Trino queries tables registered in the Hive Metastore, whose state is persisted in PostgreSQL; data files are stored as Iceberg Parquet in MinIO. Logo sources: (Trino Project 2026; HPCwire 2025; Wikimedia Commons 2026; Open Policy Agent 2026; Icon Icons 2026).*

5.3 CLI Layer

The CLI layer is the single entry point for all user interaction with TriBench. It is built on Click (Pallets 2026*a*), which handles argument parsing, nested command groups, and context passing

without global variables. The layer exposes seven command groups, each responsible for one area of the framework (Table 5.1). No framework code is imported or called outside these groups; all operations flow through CLI commands, ensuring consistent bundle resolution, logging, and error reporting regardless of which command is invoked.

Group	Responsibility	Requirements
`bundle`	Create, activate, validate, and archive bundles	**M10**, **M11**
`sys`	Start, stop, and tear down the lakehouse stack	**M8**
`data`	Generate and load datasets; manage the dataset registry	**M2**
`exp`	Run experiments and select deployment profiles	**M1**, **M3**, **M9**
`result`	Export, display, and delete stored results	**M6**, **M7**
`config`	Switch profile; Inspects configurations	**M9**
`suite`	Run grouped experiment suites end-to-end	**S1**

Table 5.1: *TriBench CLI command groups and their requirement coverage.*

For a complete reference of all sub-commands and their arguments, see Appendix A.

5.4 Bundle Abstraction

A bundle packages together the configuration, experiments, datasets, queries, and results required for a particular benchmarking study. It is self-contained and can be archived and shared for reproducibility. Table 5.2 lists the top-level elements of a TriBench bundle.

Path	Overridable	Description
`bundle.yaml`	No	Bundle name and framework version
`config/`	Yes	HOCON configuration files
`experiments/`	Yes	Experiment definition files
`apps/`	Yes	SQL query files and workload assets
`datasets/`	Yes	Static datasets and the dataset registry
`log/`	No	Execution logs produced by the framework
`results/`	No	SQLite database accumulating all run results

Table 5.2: *Top-level layout of a TriBench bundle. Overridable paths can be remapped in bundle.yaml.*

The datasets directory additionally holds a registry file that maps logical dataset names to physical file locations, decoupling experiment definitions from paths. The results directory accumulates run records across invocations with each run assigned a unique identifier so repeated executions append without overwriting (**M5**). A typical end-to-end session follows the sequence provided in Appendix B.

5.5 Configuration Layer

Each experiment in TriBench must be associated with a concrete set of configuration values covering both the deployment environment (Trino JVM heap, worker count, cluster address, backend type) and the workload (query timeout, run count, connection settings). Several problems arise with a naïve approach of configuring these manually per experiment:

Without a layered configuration model, machine-specific values (heap size, addresses, Kubernetes context) end up in experiment files; moving an experiment to a different machine requires

editing those files, and related experiments that share most parameters duplicate values manually. TriBench adopts PEEL's layered model (Section 3.1.1), expressed in HOCON (Lightbend 2026), so a profile overrides only the parameters it needs without restating the full tree. Parameters are resolved through four ordered layers, each able to override the one below it (Figure 5.3).

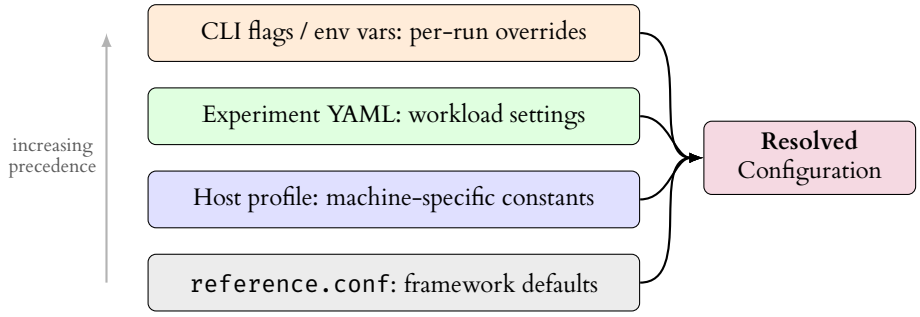


Figure 5.3: *Four-level configuration hierarchy (lowest to highest precedence). Each layer overrides keys from the one below; the resolved configuration is consumed by the orchestration layer.*

Layer 1 (reference.conf) is maintained by the framework and should not be edited. It declares every valid parameter once, so any key omitted at higher layers retains a safe default.

Layer 2 (named profile) is the recommended primary configuration point. An engineer creates one profile per deployment environment (a laptop Docker setup, a local kind cluster, a self-managed Kubernetes cluster) and stores it in `config/hosts/`. Each profile captures the machine-specific constants that should never appear in an experiment file: backend type, worker list, JVM heap, cluster address, and Kubernetes context. Profiles can live inside a bundle's own `config/hosts/` for bundle-local environments, or in the framework-level `config/hosts/` to be shared across bundles on the same machine.

Layer 3 (experiment YAML) is appropriate for parameters that vary between workloads within the same environment: run count, query timeout, or a Trino session property for one experiment. It must not contain machine-specific values; if it does, the experiment ceases to be portable across environments.

Layer 4 (CLI / environment variables) is intentionally narrow in scope. It suits one-off overrides and secret injection in CI/CD pipelines where credentials must not be stored in files. For normal benchmark use it should be avoided: a flag is not recorded in any file, making the run harder to reproduce from stored artefacts alone.

As shown in Figure 5.3, layers are combined through a recursive deep merge in priority order: nested objects are merged key-by-key rather than replaced wholesale, so a higher layer need only state the keys it changes and all others fall through from below. Named profiles at layer 2 are therefore the primary mechanism for deployment context switching and portability (**M9**, **M13**). Switching profiles switches the entire deployment context (worker count, JVM heap, backend, cluster address) without touching any experiment file. The same experiment YAML runs unchanged across local and distributed deployment environments by switching only the active host profile.

5.6 Orchestration Layer

5.6.1 Experiment Definitions

An experiment is declared as a YAML file inside the bundle's experiments directory. Each file specifies the workload independently of the deployment environment, so the same file runs

unchanged across Docker, GPG, and GKE by switching only the active host profile. Listing 5.1 shows a representative experiment definition. Queries can be referenced as SQL files in the bundle's apps directory or written as inline SQL blocks directly in the YAML, which is used for custom or exploratory workloads; a full example including inline SQL is shown in Appendix D (Figure D.8).

```yaml
name: "tpch-sf1"
system: "trino"
connection:
  host: "localhost"
  port: 8080
  catalog: "iceberg"
  schema: "tpch_sf1"
runs: 3
warmup_runs: 1
timeout_seconds: 60
monitoring:
  enabled: true
  interval_seconds: 2.0
query_files:
  - "queries/tpch/queries/q01.sql"
  - "queries/tpch/queries/q06.sql"
validation:
  min_success_rate: 0.95
```

Listing 5.1: *Experiment definition for a TPC-H SF1 run.*

The key fields and their roles are summarised in Table 5.3. The workload fields (`runs`, `warmup_runs`, `timeout_seconds`) satisfy **M3**; the `query_files` and inline `queries` blocks satisfy **M1** and **M2**; the `monitoring` block activates the subsystem described in Section 5.7 (**M4**, **S2**).

Field	Purpose
`name` / `description`	Identifier written to the results database alongside each run
`system`	Backend type; currently `trino`
`connection`	Trino coordinator address, catalogue, and schema
`runs`	Number of measured repetitions per query
`warmup_runs`	Discarded warm-up executions before measurement begins
`timeout_seconds`	Per-query execution time limit
`query_files`	Paths to SQL files inside the bundle's apps directory
`queries`	Inline SQL blocks, used for custom workloads
`monitoring`	Enables background collection and sets sampling interval
`validation`	Sets minimum success rate to pass validation

Table 5.3: *Experiment definition fields and their roles.*

5.6.2 Execution Lifecycle

Before any experiment runs, its YAML file is merged with the active HOCON hierarchy (Section 5.5); the orchestration layer consumes the resolved result without knowing which layer contributed each value (**M1**, **M9**). It exposes two execution modes: a single experiment via `tribench exp run` and a suite via `tribench suite run`. Suite mode groups multiple experiments under a shared system lifecycle so the backend starts once and tears down after the final experiment, avoiding repeated cold-start latency (**S1**). In both modes the experiment lifecycle follows five phases in order, shown in Figure 5.4. The first and last of these are carried

out by the systems layer; the orchestration layer contains no Docker- or Kubernetes-specific code, so switching backends requires only a profile change (**M13**).

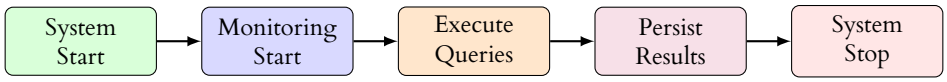


Figure 5.4: *Experiment execution lifecycle. Both single-experiment and suite modes follow these five phases in order.*

5.7 Monitoring System

The monitoring system captures hardware telemetry concurrently with query execution without blocking the query thread. Each metric source exposes a common interface with start, stop, and collect operations; a central session coordinator manages all sources and runs them in a background polling loop at the configured sampling interval. Three collectors are provided: a host resource collector that samples CPU, memory, disk I/O, and network I/O (**M4**); a Kubernetes pod collector that gathers per-pod CPU and memory and gracefully disables itself if the cluster Metrics Server is unavailable (**S2**); and a query statistics collector that reads Trino's per-query engine response after each execution (**M4**). Figure 5.5 shows the three-collector structure. The concurrent threading model and shutdown sequence are described in Section 6.7.1. A limitation of this polling approach is that resource spikes shorter than the sampling interval are not captured; for queries that complete in under one second, a single sample may represent the entire execution window.

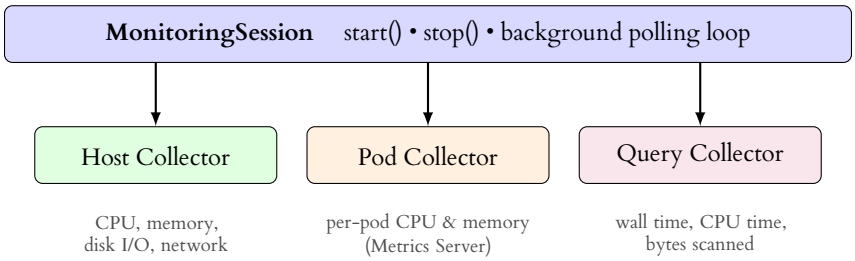


Figure 5.5: *Monitoring subsystem. The MonitoringSession drives three collectors in a shared background polling loop; each implements the same MetricCollector interface.*

5.8 Systems Layer

The systems layer manages the lifecycle of the infrastructure experiments run on. TriBench supports two interchangeable backends: Docker for single-machine deployment (**M8**) and Kubernetes for distributed clusters (**M8**, **M13**). The Kubernetes backend can target any available cluster, from a local KinD cluster for development to a managed service such as Google Kubernetes Engine, by pointing at the appropriate context in the active profile.

Both backends deploy the same four services: Trino, Hive Metastore, MinIO, and PostgreSQL. The Hive Metastore, backed by PostgreSQL, provides the Iceberg catalogue layer; it is the most widely deployed catalogue in self-managed Trino installations, so the stack reflects a common production configuration rather than a framework-specific one. MinIO acts as an S3-compatible object store, and because Trino's Iceberg connector uses S3 semantics, it provides identical connectivity to Amazon S3 without requiring cloud credentials or network access. This is what

enables local reproducibility (**M12**): the full stack runs on a single machine with no external dependencies.

Both backends expose the same lifecycle operations to the orchestration layer: set up, start, stop, and tear down. The System abstract base class defines exactly these four methods plus a status query; the orchestration layer calls only these and contains no Docker or Kubernetes API code of its own (**C1**). Figure 5.6 shows the lifecycle state machine both backends implement, driven by the following CLI commands:

```
tribench sys setup [<service>]
tribench sys start [<service>]
tribench sys stop [<service>]
tribench sys teardown [<service>]
```

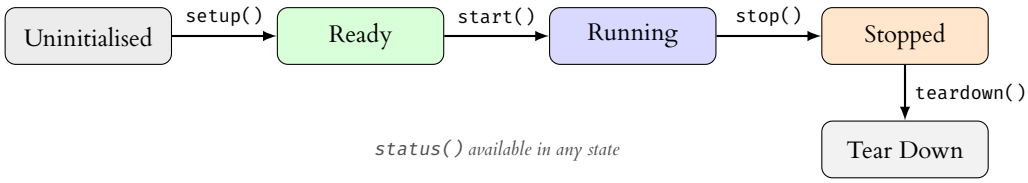


Figure 5.6: *System lifecycle state machine. Both backends implement the same four transitions.*

5.9 Dataset Generation and Loading

Before an experiment can run, benchmark data must be available as Iceberg tables in the lakehouse. TriBench separates data preparation into two sequential stages:

```
tribench data generate <dataset>
tribench data load <dataset>
```

In the generation stage, `tribench data generate` produces Parquet files from TPC-H or TPC-DS at a chosen scale factor and writes them into the bundle's datasets directory, registering each dataset in the local registry. In the loading stage, `tribench data load` reads these Parquet files through a universal three-phase pipeline: upload to MinIO, stage via a Hive external table, and copy into Iceberg via CTAS. Externally supplied datasets skip the generation stage and enter the same loading pipeline directly (**M2**). Both paths produce the same result: Iceberg tables registered in the Hive Metastore and ready to query. After loading, a provenance record captures the Iceberg snapshot identifier and row count, tying each benchmark result back to the exact data version used. The implementation of both stages is described in Section 6.5.

5.10 Result Storage

The results of every experiment run are stored in a relational database that persists across sessions. Each run produces three categories of records: per-query execution results containing timing and Trino engine statistics, a time-series of hardware telemetry samples captured during execution, and a run-level summary with overall status and resource totals. Runs are linked to a named experiment record, so executing the same workload twice appends a new run rather than overwriting the previous one. Figure 5.7 illustrates the core entities and their relationships.

The schema separates the experiment definition from its runs, so the same named workload can accumulate results across multiple invocations without duplication. Each run then owns two

independent record types — per-query execution results and time-series monitoring samples — keeping query performance data and hardware telemetry linked but separately queryable. The stored data can then be explored using the built-in analysis commands or exported for use with external tools.

SQLite (Hipp 2026) is the default database for local use: it is embedded and serverless, so the bundle requires no external database process and results are stored inside the bundle directory with no configuration. PostgreSQL (The PostgreSQL Global Development Group 2026) is supported for shared or high-volume deployments; because the storage layer uses an ORM, switching between the two requires only a configuration change. Results can then be explored via built-in `tribench res analyze` subcommands covering statistics, performance comparison, regression detection, and scalability analysis, or exported to CSV, JSON, or Parquet for external tools (**M6**, **M7**). The implementation of these commands is described in Section 6.9.

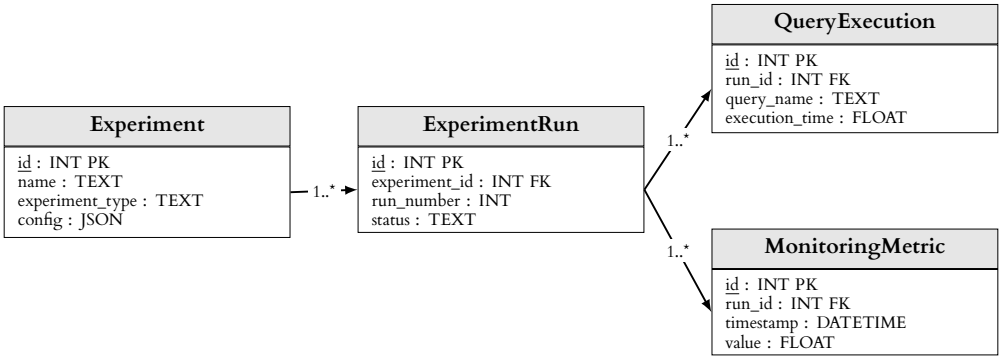


Figure 5.7: *Result storage schema (key fields only).*

5.11 Summary

This chapter covered TriBench's five-layer architecture, tracing execution from the CLI through configuration resolution and orchestration to the lakehouse services, with monitoring spanning the orchestration, systems, and data lakehouse layers, and result storage spanning the orchestration and systems layers. The next chapter details how each of these components was built in Python.

6 Implementation

Development followed an incremental strategy: the Docker backend and core execution loop were implemented first to establish a working end-to-end path, followed by the monitoring subsystem, the Kubernetes backend, and finally the bundle management system. The complete directory structure of the resulting source code repository is provided in Appendix E.

6.1 Development Approach

6.1.1 Language and Tooling

TriBench is implemented in Python 3.11. Python was chosen for a benchmarking framework targeting Trino: the official Trino client library, the Docker SDK, the Kubernetes client, and the data science ecosystem (SQLAlchemy, pandas, matplotlib) are all mature, first-class Python packages. The framework is distributed as a pip-installable package with a tribench console entry point, so a single pip install makes the CLI available globally in the active environment.

Dependencies are managed using a conda environment defined in environment.yml, chosen over a plain virtual environment because several dependencies, particularly PyArrow, require pre-compiled native binaries that conda resolves consistently across platforms. Key dependencies are Click 8 (Pallets 2026*a*), pyhocon, the official Trino Python client, psutil (Rodola 2026), the Docker SDK, SQLAlchemy 2.0 (Bayer 2026), Jinja2 (Pallets 2026*b*), pandas (McKinney 2010), and PyArrow.

6.1.2 Version Control and Testing

The project followed an iterative development methodology driven by weekly supervisor meetings rather than formal Agile sprints. To build the architecture in stable stages, Git feature branches were used to isolate major subsystems, establishing the local Docker backend and core execution loop first before introducing the monitoring, bundle management, and complex Kubernetes backends. Quality assurance relied on pytest (pytest Contributors 2026) for unit testing and KinD (Kubernetes SIGs 2026) for local Kubernetes integration testing, which allowed the full deployment pipeline to be exercised without cloud credentials.

6.2 CLI Implementation

The CLI layer is built on Click and exposes seven command groups as described in Section 5.3. Listing 6.1 illustrates how Click decorators are used to parse arguments, inject configuration, and pass the shared context object to a subcommand.

```
@click.command(name="export")
@click.argument("experiment_id")
@click.option('--format',
          type=click.Choice(['csv', 'json', 'parquet']),
          default='csv',
          help='Export format.')
@click.option('--output', type=click.Path(), help='Output file path.')
@click.option('--include-config', is_flag=True, help='Include
    experiment configuration.')
@dry_run_option
@verbose_option
@click.pass_context
def export(ctx, experiment_id, format, output, include_config, dry_run,
    verbose):
    """Export experiment results."""
    # ...
```

Listing 6.1: *Result export command demonstrating Click decorators for argument parsing, path validation, and context passing.*

Each command has access to a shared context object carrying three fields set once at startup: the active bundle path, a verbosity flag, and a dry-run flag. All subcommands read from this context rather than managing their own state, ensuring consistent bundle resolution and logging across every command.

6.3 Bundle Implementation

The bundle system centres on a manifest class that exposes the bundle's directory layout as resolved path properties. Every layer (CLI, configuration, orchestration, and result storage) calls these properties rather than constructing paths directly. This means bundles can be moved or renamed without breaking any part of the framework, as path resolution is centralised in one place. The active bundle is tracked by a state file named .tribench-bundle stored at the framework root. Locating this file follows the same upward-directory walk used by Git to find its .git directory, so commands can be run from any subdirectory within a bundle and the correct bundle is resolved automatically.

When a new bundle is created, the framework scaffolds the full directory structure and writes annotated template files: the config directory receives a commented reference profile, the experiments directory a minimal YAML template with every field documented inline, and the apps directory a placeholder queries subdirectory. Once a study is complete, bundle archive packages the bundle into a compressed tarball with the framework version pinned, so a recipient can re-run the full experiment without additional setup (**M11**).

6.4 Configuration Loading

Experiment configuration is loaded into an ExperimentConfig dataclass via a class method that applies four layers of precedence: the framework defaults in reference.conf form the base, the active host profile overrides deployment-specific parameters, the experiment YAML applies workload-specific settings, and CLI flags override last. Nested keys are merged recursively via a deep merge helper so that a profile overriding only trino.coordinator.jvm.heap does not erase the sibling keys that reference.conf provides for the same object. The result is a flat, fully-resolved object that the orchestration layer consumes without knowing which layer contributed each value.

The GCP GKE profile, for example, overrides only backend type, JVM heap, memory limits, and cluster coordinates, roughly a dozen lines against the 304 lines declared in reference.conf:

```
tribench {
  defaults.backend = "kubernetes"
  systems.trino {
    coordinator.jvm.heap = "8G"
    query.max_memory = "20GB"
    query.max_memory_per_node = "5GB"
    jvm_options = ["-server", "-XX:MaxRAMPercentage=80"]
  }
  workers = [{name = "worker-1"}, {name = "worker-2"}]
  kubernetes {
    context = "gke_tribench_us-central1-a_tribench-cluster"
    namespace = "tribench"
  }
}
```

Listing 6.2: *GCP GKE host profile, illustrating the sparse-override pattern.*

Two diagnostic commands aid debugging: tribench config show prints the fully merged parameter tree, and tribench config trace identifies which layer last set each parameter (**M14**).

6.5 Dataset Loading

Before experiments can run, benchmark data must be materialised into Iceberg tables accessible to Trino. TriBench separates this into two stages. In the generation stage, `tribench data generate` produces Parquet files for TPC-H or TPC-DS and writes them into the bundle's datasets directory. In the loading stage, `tribench data load` takes these Parquet files, whether generated or externally supplied, through a universal three-phase pipeline: upload to MinIO (Phase 1), stage through a temporary Hive external table (Phase 2), and copy into Iceberg via CTAS (Phase 3). All paths conclude with a `$snapshots` provenance record. Figure 6.1 illustrates both stages.

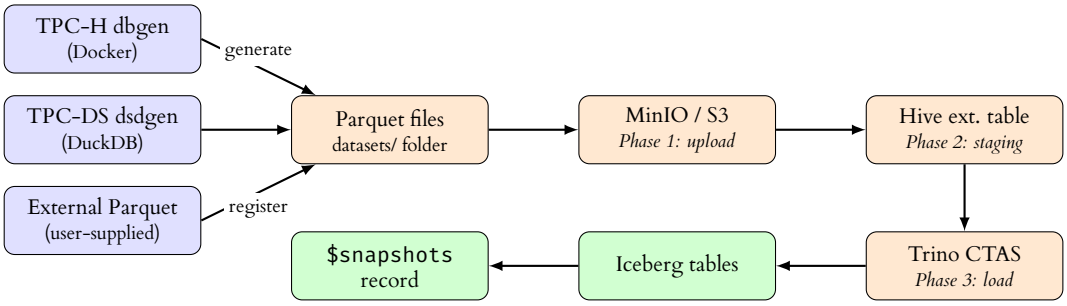


Figure 6.1: *Dataset preparation in TriBench.*

6.5.1 Dataset Generation

The `tribench data generate` command produces Parquet files for TPC-H or TPC-DS at a chosen scale factor and writes them into the bundle's datasets directory. After generation, each dataset is validated and registered in the bundle's `registry.yaml`, making it immediately available to `tribench data load` without any further configuration.

TPC-H generation is delegated to a Docker container running the official TPC-H `dbgen` tool. The container mounts the output directory and writes pipe-delimited `.tbl` files; the framework then reads each file with PyArrow. TPC-H appends a trailing delimiter to every row, which would create a spurious extra column on parsing, so the framework declares a dummy column to absorb it and drops the column after the file is read. The result is written as a Snappy-compressed Parquet file per table.

TPC-DS generation uses DuckDB (Raasveldt and Mühleisen 2019), an embeddable in-process analytical database, rather than building `dsdgen` from source, avoiding cross-platform compilation issues. The framework opens an in-process DuckDB connection, installs and loads the built-in TPC-DS extension, calls `dsdgen` at the requested scale factor, then exports each of the 24 tables to Parquet directly via a `COPY ... TO` statement.

Once generation completes, the Parquet files in the datasets directory are passed to the universal loader described in Section 6.5.2.

6.5.2 Universal Loader

All Parquet datasets, whether generated by `tribench data generate` or supplied externally, are loaded through the same benchmark-agnostic three-phase pipeline implemented in the UniversalIcebergLoader class, satisfying requirement **M2**.

Phase 1 – Parquet upload. Each Parquet file is copied to MinIO under a schema-namespaced prefix using the MinIO client. The loader checks whether the file is already present before uploading, so re-running a partially failed load is safe.

Phase 2 – Hive external table staging. Trino cannot read raw Parquet files directly from S3 without a table definition. The loader creates a temporary external table in the Hive Metastore pointing at the uploaded file. Column definitions are derived automatically by reading the PyArrow schema embedded in the Parquet file footer and mapping each Arrow type to its Trino SQL equivalent. The staging table receives a millisecond timestamp suffix to prevent conflicts when two loads run concurrently.

Phase 3 – CTAS into Iceberg. The loader issues a CTAS statement that selects all rows from the Hive staging table and writes them into the Iceberg catalog. Optional partition columns, passed as a per-table mapping, are translated into an Iceberg partitioning array in the WITH clause. The staging table is dropped unconditionally in a finally block after each table completes.

```
CREATE TABLE iceberg.benchmark.lineitem
WITH (
  format = 'PARQUET',
  partitioning = ARRAY['month(l_shipdate)'],
  location = 's3://warehouse/benchmark/lineitem/'
)
AS SELECT * FROM hive.staging.lineitem_1710000000000;
```

Listing 6.3: Universal loader CTAS pattern for copying Hive staging to Iceberg

After all tables are loaded, the loader queries the Iceberg `$snapshots` metadata table for each table and records the resulting snapshot identifier and commit timestamp alongside the row count. This lightweight provenance record ties each benchmark result back to the exact data version that produced it, supporting reproducibility across runs.

6.6 Experiment Execution

6.6.1 Single Experiment

The TrinoExperiment class implements the five-phase lifecycle described in Section 5.6.2. Within the execute queries phase, each query first runs for a configurable number of warmup iterations whose output is discarded, ensuring that JVM class loading and Iceberg metadata caching do not bias the measured results (Georges et al. 2007). Measured runs then proceed sequentially using the Trino Python client's cursor interface.

After each query, engine statistics are read from the cursor's stats attribute before the next execute call, as each call overwrites the previous stats. Listing 6.4 shows the relevant extraction:

```
cursor.execute(query)
rows = cursor.fetchall()
# Stats must be read before the next execute() call
stats = cursor.stats or {}
result = QueryResult(
   wall_time_ms = stats.get("elapsedTimeMillis"),
   cpu_time_ms = stats.get("cpuTimeMillis"),
   bytes_scanned = stats.get("processedBytes"),
   peak_memory = stats.get("peakMemoryBytes"),
   spilled_bytes = stats.get("spilledBytes"),
)
```

Listing 6.4: *Extracting Trino engine statistics from the cursor immediately after fetchall.*

6.6.2 Suite Execution

When running a suite, the ExperimentSuite class groups multiple experiments under a shared system lifecycle, passing the already-running system reference to each TrinoExperiment so it connects without triggering another start (**S1**). If any experiment fails, the suite logs it, continues with the remaining experiments, and reports a partial success rather than aborting.

6.6.3 Parallel Query Execution

For workloads that issue multiple queries concurrently, TriBench provides a parallel execution mode built on a thread-safe connection pool. The pool is backed by a `queue.Queue` and supports both eager pre-creation and lazy creation, configurable per experiment. Each worker thread in a `ThreadPoolExecutor` acquires a connection from the pool, executes its assigned query, releases the connection back to the pool, and appends its result to a shared list under a `threading.Lock`. Results are collected as they complete using `as_completed`, so a slow query on one thread does not delay the recording of fast queries on others. This design keeps the connection count bounded by the pool size regardless of the degree of parallelism, preventing connection exhaustion on the Trino coordinator.

6.7 Concurrent Monitoring

6.7.1 Threading Model

The monitoring subsystem runs in a background thread so resource sampling never blocks query execution. A MonitoringSession manages all collectors via a shared stop flag; the main thread spawns the background collector at session start, then sets the stop flag after all queries complete

and calls `thread.join()` to wait for the final loop iteration to finish before flushing buffered samples to the database. Figure 6.2 shows the two-thread interaction.

```
def _collection_loop(self):
    while not self._stop_event.is_set():
        with self._metrics_lock:
            self._collected_metrics.extend(self.collect_metrics())
        self._stop_event.wait(self.config.interval_seconds)
```

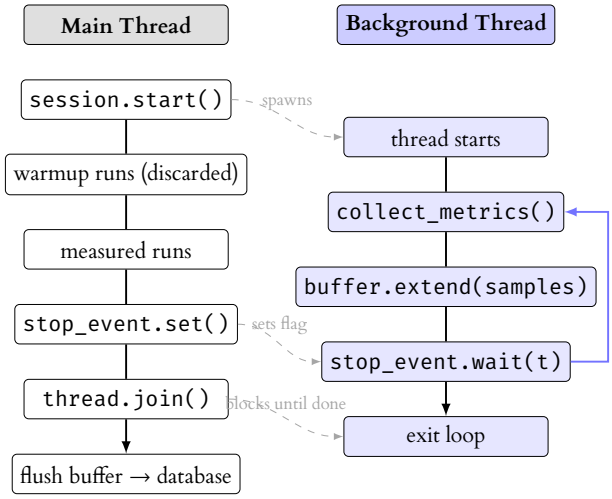


Figure 6.2: Monitoring thread lifecycle.

6.7.2 Host Resource Collector

The host resource collector uses psutil to sample CPU utilisation, memory, disk I/O, and network I/O at each interval. A key implementation detail is that psutil reports disk and network counters as cumulative totals since system boot. To produce per-experiment deltas, the collector records baseline counter values at session start and subtracts them from each subsequent reading:

$$\Delta_{\text{disk_read}} = C_{\text{now}} - C_{\text{baseline}}$$

Without this baseline subtraction, the reported figures would reflect the lifetime of the host process rather than the experiment duration. CPU utilisation does not require baseline subtraction because psutil already computes a percentage over the elapsed interval.

6.7.3 Kubernetes Pod Collector

On Kubernetes deployments, a second collector queries the cluster Metrics Server for per-pod CPU and memory at each interval. The collector issues a kubectl top pods command via subprocess and parses the output into normalised per-pod records, converting millicores and mebibytes into consistent numeric units. Subprocess was preferred over the Kubernetes Python metrics client because kubectl top is available on any cluster where the Metrics Server is deployed and avoids an additional client dependency. If the Metrics Server is unavailable, for instance because it was not deployed to the cluster, the collector catches the error at startup and disables itself gracefully, allowing the experiment to continue with host-level metrics only.

6.8 Deployment Backends

6.8.1 Docker Backend

The Docker backend generates Trino configuration files from Jinja2 templates using the resolved profile; Jinja2 was chosen because its variable substitution maps directly to the resolved HOCON profile keys, avoiding a custom rendering step. Changing the JVM heap and restarting immediately produces an updated coordinator config. Once the containers are started, the backend performs a two-stage readiness check: the first stage polls the Docker SDK until each container reports running; the second polls the Trino coordinator's `/v1/info` REST endpoint and checks that the `starting` field has returned false, confirming the JVM is fully initialised before queries are submitted.

6.8.2 Kubernetes Backend

The Kubernetes backend follows the same Jinja2 template approach, generating service manifests from the active profile and applying them to the cluster via kubectl. Services are started in dependency order (MinIO first, then PostgreSQL, then Hive Metastore, then Trino), with each stage waiting for the previous to reach a ready state before proceeding. This ordering was important in practice: applying all manifests simultaneously caused Hive Metastore to fail on startup because the PostgreSQL catalogue it depends on was not yet accepting connections.

Once the cluster is running, the backend establishes a port-forward tunnel from a fixed local port to the Trino coordinator service. The tunnel runs as a background subprocess managed through a PID file, so it persists independently of the terminal session that started it. If the tunnel process dies, for instance due to a network drop, it can be restarted independently with `tribench sys port-forward start`without touching the cluster. However, if the tunnel dies silently mid-experiment, the orchestration layer reports a Trino connection error rather than identifying the tunnel as the cause, requiring manual diagnosis. The orchestration layer connects to the same local address regardless of whether the backend is Docker or Kubernetes, which is the concrete mechanism behind deployment portability (**M13**).

6.9 Result Storage

6.9.1 Persistence

Results are persisted using SQLAlchemy's ORM, which abstracts over both SQLite and PostgreSQL. Each invocation appends a new run record rather than overwriting existing data, so repeated runs accumulate and earlier results are never lost (**M5**). Each run is assigned a unique identifier combining the experiment name, a UTC timestamp, and a short random hex suffix to prevent collisions.

6.9.2 Export

Results can be exported in three formats via tribench res export. CSV produces two files per run: one for query execution records and one for monitoring metric samples, with a shared timestamp in the filename to keep them paired. JSON packages all records alongside the full resolved configuration snapshot into a single document, making it self-contained for archiving. Parquet targets downstream analysis with tools such as pandas or Apache Spark, where the columnar format allows efficient filtering over large numbers of runs (**M6**, **M7**).

6.9.3 Built-in Analysis

To support rapid exploratory analysis without requiring an external export step, the framework provides five core analytical capabilities that operate directly against the result database. These routines are implemented using Python's standard `statistics` module, `numpy` for efficient percentile calculations, and `scipy.stats` for rigorous significance testing (e.g., confidence intervals and two-sample t-tests). The supported analyses include:

- **Descriptive Statistics:** computations across all runs for each query (mean, std dev, percentiles), incorporating anomaly identification via IQR-based outlier detection (Jain 1990).
- **Performance Profiling:** reports on throughput (QPS), total execution time, and per-query latency summaries for a single experiment run.
- **Head-to-Head Comparison:** evaluates differences between experiment runs using a two-sample t-test (Georges et al. 2007) to determine if latency variations are statistically significant.
- **Regression Detection:** identifies statistically significant slow-downs between runs, classified by severity. This requires both a configurable threshold and a t-test confirmation, making it robust for CI pipelines where false alerts must be minimised.
- **Scalability Assessment:** calculates speed-up and parallel efficiency per query (Amdahl 2000) between runs at different worker counts to quantify scaling behavior.

Example output from all five subcommands is shown in Appendix D.

6.10 Extending TriBench

TriBench is designed so that its core components can be swapped or extended with minimal changes.

New deployment backends. All deployment logic sits behind a System abstract base class with four lifecycle methods (setup, start, stop, teardown) plus a status query (Section 6.8). Adding a backend such as Docker Swarm requires only a concrete subclass implementing these methods; the orchestration layer calls nothing else (**C1**).

Alternative stack components. PostgreSQL and MinIO are themselves System implementations rendered from Jinja2 templates. Replacing either, for example substituting MySQL for PostgreSQL or pointing at a managed S3 bucket instead of MinIO, requires a new subclass, a matching template, and the relevant configuration keys in `reference.conf`. Inter-service addresses are resolved from configuration at render time, so the rest of the stack adapts automatically.

New monitoring sources. The monitoring subsystem exposes a MetricCollector interface with three methods: start, stop, and collect (Section 6.7). Any metric source that can be polled from Python can be integrated by implementing these three methods and registering the collector with the monitoring session. Collectors can disable themselves gracefully if the source is unavailable, as the Kubernetes pod collector already does when the Metrics Server is absent.

6.11 Summary

This chapter covered the key implementation decisions across each component: the four-layer HOCON merge, cursor-attached stats extraction, the two-stage Docker readiness check, dependency-ordered Kubernetes startup, and the five built-in analysis commands. Together these satisfy all Must-Have and Should-Have requirements, as confirmed in Chapter 7.

7 Experiments

This chapter presents six experiments evaluating TriBench: unit testing, three deployment scenarios, a cross-environment portability study, a framework overhead measurement, and a reproducibility analysis.

7.1 Unit Testing

A suite of 267 unit tests was written using pytest pytest Contributors (2026), covering configuration, storage, query execution, statistical analysis, monitoring, CLI utilities, and bundle management. All external dependencies (Trino, Docker, Kubernetes, and PostgreSQL) are replaced with stubs so that tests run without any deployed infrastructure, with the result database held in memory. All 267 tests pass.

Figure D.7 in Appendix D shows line coverage per module across the tested components, giving 85% overall, above the 70% threshold recommended by Microsoft Microsoft (2025). Five modules reach 100%. The lower figures for query execution and monitoring reflect error-handling paths that only trigger under live network failures or when a background thread is forcibly interrupted; these are exercised by the end-to-end experiments in the sections that follow.

7.2 Experiment Methodology

This chapter takes a demonstration-based approach: each experiment shows that TriBench operates correctly and meets its stated requirements. Workloads are drawn from TPC-H and TPC-DS, chosen because they are industry-standard, vendor-neutral benchmarks with well-defined correctness criteria and reproducible data generation (Huppler 2009). Each query is preceded by a configurable number of warmup runs whose results are discarded, as described in Section 6.6.1. Measured runs are repeated and recorded independently so that variability across runs is visible. Six experiments are presented: three deployment scenarios (local Docker, on-premises Kubernetes, and GKE cloud), a cross-environment portability study, a framework overhead measurement, and a reproducibility analysis. Sections 7.7–7.9 evaluate portability, overhead, and reproducibility across all three environments; in each case the same deployment configurations are used as established in Sections 7.4–7.6: a single-worker Docker stack, a two-worker University of Glasgow Kubernetes cluster, and a four-worker GKE cluster. Readers can recreate these experiments by following the guidance in Appendix B.

7.3 Experimental Setup

7.3.1 General Experiment Workflow

Because TriBench abstracts deployment and execution logic, the core workflow is identical across all environments. All benchmark datasets were prepared using TriBench's own data pipeline; custom datasets skip the generation step and are supplied as Parquet files directly. The only

differences between environments lie in system connectivity and profile selection. The standard procedure used for all experiments is:

```
tribench bundle create <bundle_name>
tribench config profile set <profile_name>
tribench sys start all
tribench data generate --suite <suite> --scale-factor <sf>
tribench data load <dataset_name>
tribench exp run <experiment.yaml>
```

This uniform sequence scaffolds a self-contained directory, activates the target environment, boots the full TriBench stack (Trino, Hive Metastore, MinIO, PostgreSQL), registers the required tables in Iceberg, and triggers the automated benchmark. The following sections detail the specific deployment configurations and the unique challenges addressed in each environment, omitting these repetitive workflow steps.

7.3.2 Local Docker Environment

The local experiments run on an Apple MacBook Pro with an M4 Pro chip, 24 GB unified memory, and an NVMe SSD running macOS Sequoia. Docker Desktop provides the container runtime, configured with a 4 GB coordinator heap. The TPC-H scale factor 1 dataset was generated and loaded into the Iceberg catalogue. The entire stack is brought up with a single command; step-by-step setup instructions are provided in Appendix B. All experiment definitions, configuration, datasets, and results are packaged into a single bundle for portability and reproduction.

7.3.3 On-Premises Kubernetes Environment

The On-Premises Kubernetes experiments run on the University of Glasgow's GPG Kubernetes cluster. Each physical node runs a single Kubernetes node, so each Trino worker pod has exclusive access to a dedicated machine with no resource sharing. Each node has dual Intel Xeon E5-2640 processors (16 cores at 2 GHz), 64 GB RAM, 300 GB local disk, and a 10 Gb Ethernet interconnect. TriBench connects to the cluster automatically using port forwarding, and the Kubernetes Metrics Server is deployed to collect per-pod CPU and memory usage. A two-worker configuration is used with a 16 GB coordinator heap. The TPC-DS scale factor 10 dataset was generated and loaded alongside a custom domain dataset, both registered in the bundle.

7.3.4 GKE Cloud Environment

The GKE experiments run on a Google Kubernetes Engine cluster in the us-central1-a region. Each node is a 4-vCPU, 16 GB RAM instance on Google's Arm-based Tau T2A platform. GKE is chosen for the scaling study because nodes can be added and removed without manual cluster reconfiguration, making it straightforward to reproduce each worker count cleanly. The study sweeps one, two, and four workers on a fixed set of TPC-H queries at scale factor up to 100; a subsequent suite run uses four workers throughout. GKE's built-in Metrics Server provides per-pod telemetry. After all runs, the bundle is archived so that the full experiment can be reproduced from a single artefact.

7.4 Local Docker Deployment

This section evaluates TriBench on a local Docker environment (Scenario 1, Section 4.1).

7.4.1 Deployment and Configuration

TriBench requires no manual Docker Compose authoring for this scenario. Following the standard workflow from Section 7.3.1, a dedicated bundle was created and pointed at the built-in `docker` profile.

The `docker.conf` profile is included in the TriBench distribution and configures a Trino coordinator with a 4 GB JVM heap, Hive Metastore backed by PostgreSQL, and MinIO as the object store. The full stack (five containers: Trino coordinator, one worker, Hive Metastore, MinIO, and PostgreSQL) reaches a ready state in under 90 seconds. The TPC-H SF 1 dataset was then loaded into the Iceberg catalogue.

No additional configuration was required. This demonstrates that TriBench's Docker backend handles the full lifecycle (network creation, container scheduling, port binding, and health checking) without user intervention (**M8**).

7.4.2 Experiment Execution

The experiment was defined in `tpch-all.yaml`, specifying all 22 TPC-H queries, three measured runs, one warmup run, a 60-second per-query timeout, and a 95% minimum success rate threshold. It was launched with a single command:

```
tribench exp run tpch-all.yaml
```

All 22 queries completed successfully across all three measured runs (66 executions in total). Mean query latency ranged from 0.17 s (Q6, filter-only) to 0.60 s (Q13, aggregation with a correlated subquery). The three slowest queries, Q13 (0.60 s), Q21 (0.53 s), and Q8 (0.44 s), are all join-heavy or involve large intermediate result sets, consistent with expected TPC-H behaviour at SF 1. Figure 7.1 shows the mean execution time for all 22 queries.

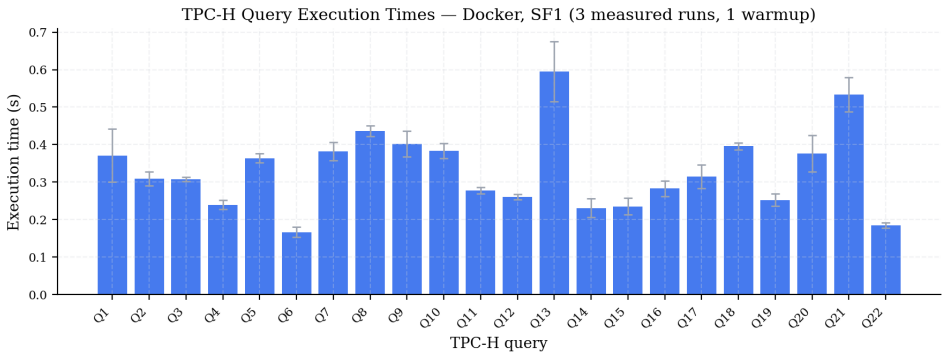


Figure 7.1: Mean TPC-H query execution times on Docker (SF 1, 3 measured runs).

7.4.3 Host Monitoring

With `monitoring.enabled = true`, TriBench's background collector polled host CPU and memory at a 2-second interval throughout the run, recording each sample against its wall-clock timestamp. Query boundaries were recorded automatically so that per-query resource windows can be reconstructed from the results database. Figure 7.2 shows CPU utilisation over the duration of the experiment, with vertical lines marking the start of each query group.

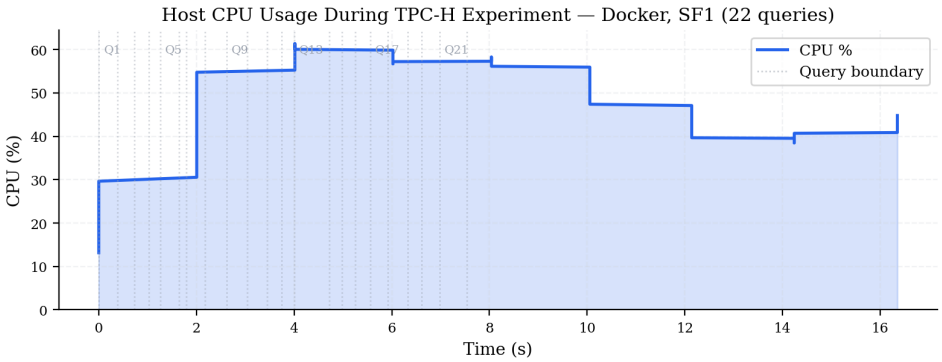


Figure 7.2: Host CPU utilisation during the Level 1 TPC-H experiment (Docker, SF 1, Gaussian-smoothed). Dotted vertical lines mark individual query boundaries. CPU rises and plateaus as heavier queries execute, then steps down toward the end of the workload.

CPU utilisation rises from approximately 15% at experiment start to a sustained plateau of 55–60% as the heavier join and aggregation queries execute, before stepping down as the remaining lighter queries complete. The query boundary markers confirm that monitoring samples span the full experiment duration, providing continuous telemetry throughout the workload (**M4**).

7.4.4 Result Storage and Export

Results were stored automatically in the bundle's SQLite database. Re-running the experiment a second time without clearing the database appended a new `ExperimentRun` record under the same experiment, confirming that results accumulate across invocations rather than being overwritten (**M5**); example output from `tribench res show` is shown in Appendix D. Results were then exported to CSV for downstream analysis (**M6**):

```
tribench res export <experiment_id> --format csv
```

This section provides evidence for requirements **M1** (declarative YAML experiment definition), **M2** (TPC-H benchmark support), **M3** (execution control), **M4** (telemetry collection), **M5** (structured non-overwriting result storage), **M6** (data export), and **M8** (Docker lifecycle management).

7.5 On-Premises Kubernetes Deployment

This section evaluates TriBench on the University of Glasgow GPG Kubernetes cluster (Scenario 2), running TPC-DS and a custom domain dataset.

7.5.1 Kubernetes Deployment

Following the standard workflow, Level 2 uses a dedicated bundle created separately from the Level 1 Docker bundle to maintain context isolation.

The cluster nodes are on a private network and are not directly accessible from a laptop. Connectivity was established by opening an SSH tunnel from the local machine to the control plane node (`gpgnode-13`), forwarding the Kubernetes API server port:

```
ssh -L 6443:localhost:6443 gpgnode-13.dcs.gla.ac.uk -N
```

The kubeconfig was copied from the control plane, patched to route API calls through `localhost:6443`, and merged into the local kubeconfig under the context name `gpg-multinode`. With the tunnel active, the local `kubectl` and TriBench treat the remote cluster identically to a local one.

A host profile `gpg-multinode.conf` was created following the same sparse-override pattern as Listing 6.2, differing only in the coordinator heap (16 GB to match the GPG node RAM) and the cluster context (`gpg-multinode` in place of the GKE context name).

With the profile activated, TriBench applied the Kubernetes manifests, distributed pods across nodes (the coordinator and infrastructure pods to `gpgnode-13`, Trino workers to `gpgnode-14` and `gpgnode-15`), waited for all pods to reach a ready state, and established a `kubectl port-forward` tunnel to `localhost:8080` for query submission, all without manual `kubectl` interaction. This demonstrates transparent Kubernetes lifecycle management (**M8**) and that the same CLI workflow operates across backends without modification (**M13**).

7.5.2 TPC-DS Workload

The `tpcds-gpg` experiment ran ten representative TPC-DS queries against the TPC-DS SF 10 dataset loaded into the cluster's Iceberg catalogue. Five measured runs were collected after one warmup run. All 50 query executions completed successfully. Figure 7.3 shows the mean execution time per query.

Q13, a multi-way join across six TPC-DS tables, was the slowest query at a mean of 3.47 s. Q3 and Q52, both date-range aggregations over store sales, were the fastest at under 1 s. Variance across the five runs was low for most queries, consistent with a stable cluster state. This confirms TPC-DS benchmark support on the Kubernetes backend (**M2**).

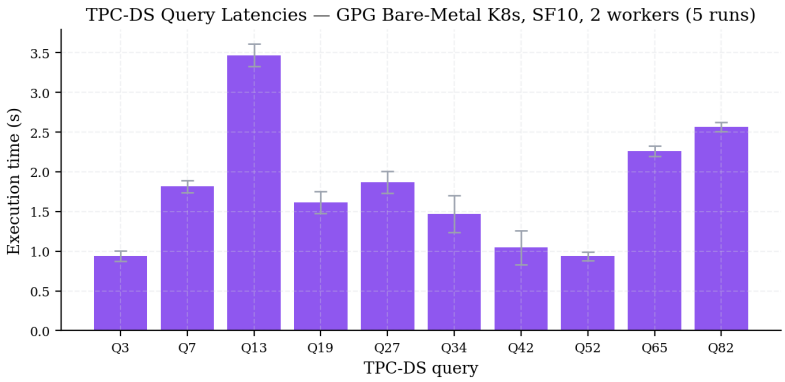


Figure 7.3: *Mean TPC-DS query execution times on the University of Glasgow GPG cluster (SF 10, 2 workers, 5 measured runs). Error bars show one standard deviation across runs.*

7.5.3 Custom Dataset

The `custom-gpg` experiment demonstrates that TriBench is not restricted to TPC workloads. A synthetic e-commerce dataset was used, comprising four related Parquet files: customers (1,000 rows), products (500 rows), orders (5,000 rows), and order_items (15,000 rows). Existing public e-commerce datasets were considered but rejected because those available consisted of a single flat Parquet file rather than a normalised multi-table schema; a multi-table schema was necessary to evaluate realistic join workloads. The dataset was therefore generated synthetically, registered in the bundle, and loaded into the Iceberg catalogue.

Two queries were then defined directly as inline SQL within the experiment file, without needing separate `.sql` files; the full experiment definition is shown in Appendix D (Figure D.8).

Both queries completed successfully across three measured runs with mean latencies of 0.59 s and 0.42 s respectively. This demonstrates support for user-supplied datasets and custom queries (**M2**). The dataset was intentionally small at 15,000 rows, so the universal loader's throughput at production-scale files is not exercised here.

Monitoring was active throughout; figures are deferred to Section 7.6 where the GKE deployment provides richer pod-level telemetry. This section provides evidence for **M2**, **M8**, and **M9**; cross-environment portability (**M13**) is evaluated in Section 7.7.

7.6 GKE Cloud Deployment

Level 3 evaluates TriBench on Google Kubernetes Engine, covering a worker scaling study and a full suite run at four workers.

7.6.1 GKE Deployment

Following the standard workflow, a dedicated bundle was created for the Level 3 experiments.

Unlike the local Docker and on-premises environments, GKE requires the Google Cloud CLI (`gcloud`) to be installed and authenticated on the local machine. The GKE cluster was provisioned via the Google Cloud Console and credentials were fetched with:

```
gcloud container clusters get-credentials tribench-cluster \
  --zone us-central1-a --project tribench
```

This command writes the cluster credentials into the local kubeconfig under the context `gke_tribench_us-central1-a_tribench-cluster`, which TriBench references directly. No SSH tunnel is required, as the GKE API server is publicly reachable via Google's managed endpoint. The host profile `gcp-gke-4w.conf` follows the same sparse-override pattern as Listing 6.2, extended to four workers with an 8 GB coordinator heap. With the profile activated, TriBench deployed the full stack to the GKE cluster identically to the GPG deployment: same manifests, same lifecycle commands, different context.

7.6.2 Worker Scaling Study

To evaluate how TriBench handles multi-configuration experiments and how Trino scales across worker counts, the full TPC-H SF 1 workload (all 22 queries, 3 measured runs) was executed at one, two, and four workers. Only the `workers` list in the host profile was changed between runs; the experiment YAML, dataset, and query files were identical. Figure 7.4 shows mean query latency at each worker count.

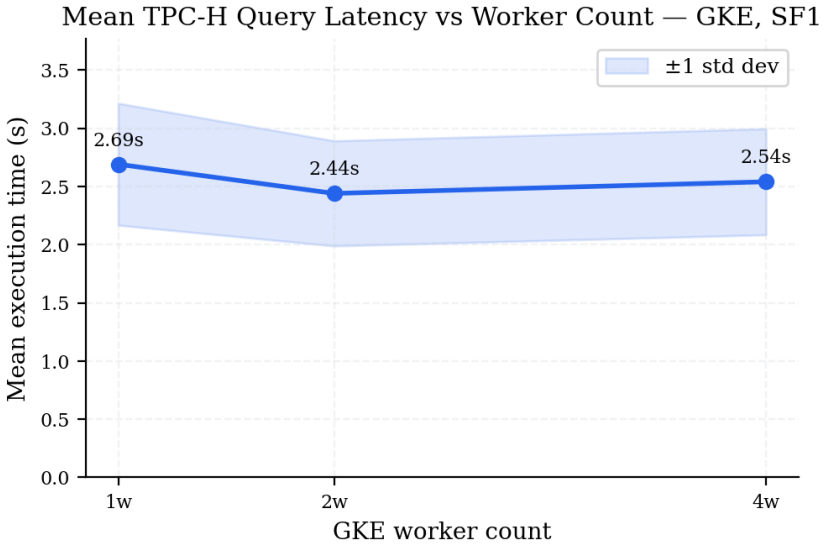


Figure 7.4: *Mean TPC-H query latency on GKE at one, two, and four workers*

Figure 7.4 shows mean query latency decreasing as workers are added across the three configurations. TriBench managed the full stack lifecycle between each worker count automatically, tearing down, reconfiguring, and restarting without manual intervention, and collected per-query telemetry throughout each run. Speed-up and parallel efficiency were then computed using `tribench res analyze scalability`, demonstrating the end-to-end scaling study workflow (**M7**); example output is shown in Appendix D.

7.6.3 Full Suite at Four Workers

With four workers fixed, a full benchmark suite was executed using:

```
tribench suite run gke-suite.yaml
```

The suite comprised nine experiments run sequentially, including overhead measurement runs, TPC-H at four scale factors (SF tiny, SF 1, SF 10, SF 100), TPC-DS selected queries, and a custom e-commerce workload. TriBench managed the stack lifecycle between experiments automatically, starting the system before the first experiment and stopping it after the last, with no user interaction required throughout. The SF 100 experiment is excluded from Figure 7.5 and shown as an annotation due to its long duration to keep the remaining bars readable.

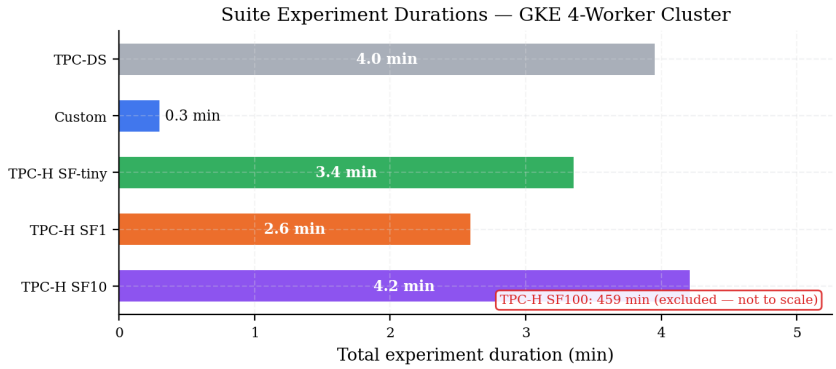


Figure 7.5: Suite execution timeline on GKE 4w (SF 100 excluded, shown as annotation).

Throughout the suite, TriBench's Kubernetes monitoring collected per-pod CPU telemetry via the GKE Metrics Server. Figure 7.6 shows the breakdown for the SF 1 run. The coordinator handles planning and dispatch (orange) while the four workers (blue) shoulder the bulk of the compute, peaking at around 2,000 millicores combined from Q13 onwards as the workload shifts to join-heavy queries. This provides direct evidence for per-pod Kubernetes metric collection (**S2**).

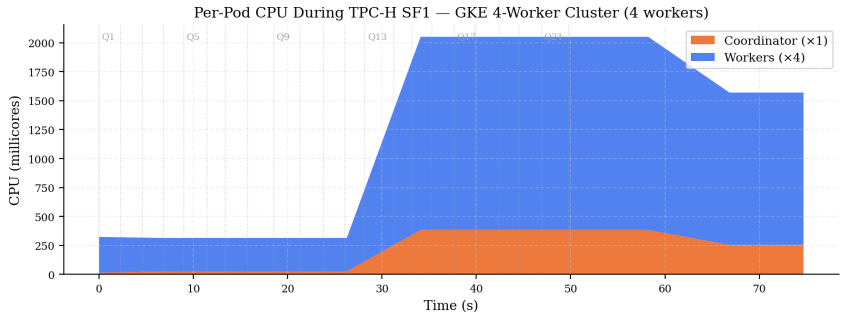


Figure 7.6: Per-pod CPU usage during TPC-H SF 1 on GKE (1 coordinator, 4 workers).

This section provides evidence for requirements **S1** (suite automation and unattended execution) and **S2** (per-pod Kubernetes metrics).

7.6.4 TPC-H SF100 Query Performance

SF 100 represents an industry-scale dataset: approximately 600 million lineitem rows, 100 GB of raw data, processed by a distributed Trino cluster on cloud infrastructure. Running TPC-H at this scale is representative of production data warehouse workloads and is a scale factor used in published systems benchmarking studies (Transaction Processing Performance Council 2026*b*; Camacho-Rodríguez et al. 2024). The fact that TriBench executed the full 22-query suite at SF 100 unattended, collected per-query telemetry throughout, and stored reproducible results demonstrates that the framework is not limited to small-scale experimentation; it is suitable for the kind of sustained, large-scale benchmarking carried out in industrial and academic systems research. The 7.7-hour duration does expose a practical limitation, however: TriBench provides no mechanism to estimate query runtime before committing to a full run, which makes scheduling large-scale experiments difficult without prior knowledge of the workload. Integrating a real-

time visibility layer as discussed in Section 8.3 would allow practitioners to monitor progress and abort runs that are clearly over-budget before they complete.

Figure 7.7 shows the mean execution time per query across three measured runs with all 22 queries completed successfully. Mean latencies range from 46 s (Q16, a parts-based filter query) to 842 s (Q15, a view-based query that identifies the top revenue supplier over the full lineitem table). The variation across queries reflects the well-documented spread in TPC-H complexity at scale, and tight error bars on most queries confirm that results remain reproducible even at SF 100 (**M12**). Across 109,844 monitoring samples collected during the SF 100 run, the coordinator averaged 39.6 millicores while workers averaged 144.9 millicores, confirming that execution load distributes across workers as expected.

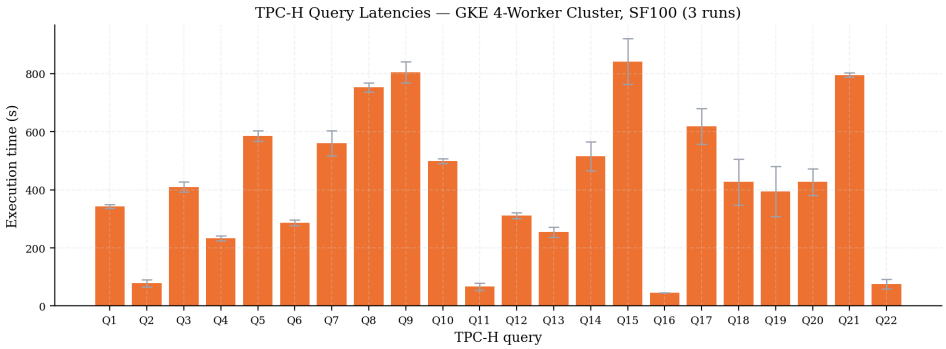


Figure 7.7: *Mean TPC-H query execution times on GKE 4w at SF 100 (3 measured runs, 1 warmup).*

This section provides evidence for **M12** (reproducibility at scale).

7.7 Cross-Environment Portability

A portability anchor experiment ran five TPC-H queries at SF 1 (the same YAML, dataset, and query files used across all three deployment experiments) on Docker, the University of Glasgow GPG cluster, and GKE. The only difference between runs was the active host configuration file. Table 7.1 compares mean execution times.

Table 7.1: *Mean execution time (seconds) for TPC-H portability anchor queries across all three deployment environments (SF 1, identical experiment YAML). Only the host configuration file differed.*

Query	Docker 1w (s)	GPG 2w (s)	GKE 4w (s)
Q1	0.347	1.016	2.164
Q6	0.174	0.684	1.899
Q9	0.546	1.431	2.878
Q18	0.476	1.661	2.613
Q21	0.587	2.024	3.147

Latencies increase from Docker through GPG to GKE, reflecting differences in both hardware and network topology. The Docker environment runs on an M4 Pro MacBook Pro querying over the loopback interface, while the GKE nodes are 4-vCPU Arm Tau T2A instances connected via Google's virtual private network; GPG routes through a university LAN with Calico overlay networking. Both factors contribute to the observed differences, so results across environments are not directly comparable without accounting for them. All three environments produced valid,

complete query results from identical experiment definitions without modification, demonstrating cross-environment portability (**M13**).

7.8 Framework Overhead

To confirm that background monitoring does not materially perturb query execution (**S3**), an instrumented run (monitoring enabled, 1-second polling) was compared against an uninstrumented run across all three environments using Q1, Q6, and Q18 at SF 1, with five measured runs per configuration after one warmup. Table 7.2 reports the results; Figure 7.8 summarises mean overhead per environment.

Table 7.2: *Framework overhead: mean execution time for instrumented and uninstrumented runs. Values marked as noise are within run-to-run variance.*

Environment	Query	Instrumented (s)	Uninstrumented (s)	Overhead (%)
Docker 1w	Q1	0.445	0.417	+6.7
	Q6	0.117	0.115	+2.1
	Q18	2.042	1.994	+2.4
GPG 2w	Q1	0.953	0.943	+1.1
	Q6	0.397	0.424	$<\pm5$ (noise)
	Q18	1.857	1.920	$<\pm5$ (noise)
GKE 4w	Q1	2.907	3.012	$<\pm5$ (noise)
	Q6	1.877	1.924	$<\pm5$ (noise)
	Q18	5.202	5.543	$<\pm5$ (noise)

Mean overhead is below 5% across all environments (**S3**), averaging 3.8% on Docker and within noise on both Kubernetes clusters. Docker's Q1 individually reaches 6.7% because psutil's fixed polling rate is proportionally costlier for sub-second queries. On GPG and GKE, longer query times mean the overhead signal falls below run-to-run variance. The Metrics Server API call for per-pod collection (**S2**) adds no statistically distinguishable cost on either cluster. A comparison against an ad-hoc script baseline was not made: faithfully replicating TriBench's monitoring, warmup, and result persistence for a fair comparison would require significant effort across all three environments.

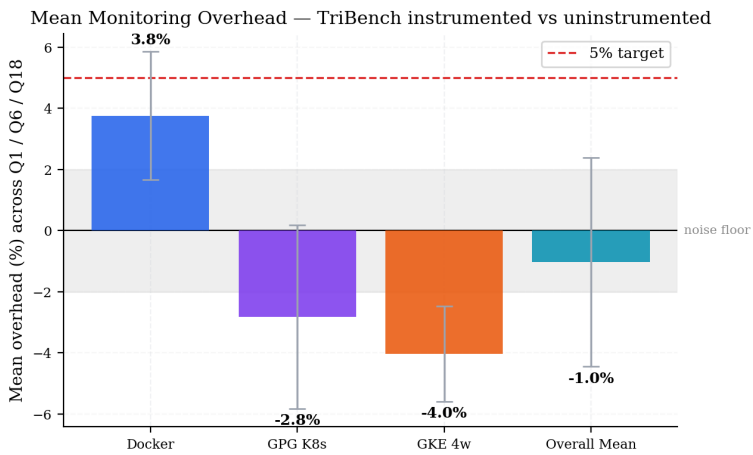


Figure 7.8: *Mean monitoring overhead per environment. The dashed line marks the 5% target (**S3**).*

7.9 Reproducibility

Section 2.4 identified four dimensions for reproducible benchmarking. Table 7.3 provides evidence for **execution control**: the coefficient of variation (CV = $\sigma/\mu \times 100\%$) across repeated runs for representative queries in each environment.

Table 7.3: *Run-to-run CV across all queries per environment. CV = $(\sigma/\mu) \times 100\%$.*

Environment	Queries	Runs	Mean CV (%)	≤10%
Docker SF 1 (TPC-H)	22	3	7.2	17/22
GPG SF 10 (TPC-DS)	10	5	7.9	8/10
GKE SF 100 (TPC-H)	22	3	8.1	16/22

Mean CV is below 10% across all three environments, with most queries falling well within that threshold. Residual variability is expected given the different underlying platforms: sub-second queries on Docker are susceptible to OS scheduling noise, while GKE introduces hypervisor scheduling jitter (Jain 1990)

Environment specification. The portability experiment (Section 7.7) demonstrates this: the same experiment YAML produced valid results on all three platforms with only the host configuration file changed.

Data consistency. After each load, TriBench records the Iceberg snapshot identifier and row count for every table alongside each run record, making silent data changes detectable (Figure D.9 in Appendix D).

Measurement capture. The overhead experiment (Section 7.8) shows background monitoring perturbs execution times by less than 5% on average, so monitoring can remain enabled without affecting measured latencies (**S3**).

7.10 Evaluation Limitations

Several factors bound the interpretation of these results. The GKE environment share infrastructure with other workloads; warmup runs and three measured repetitions mitigate transient noise but cannot eliminate it entirely. Iceberg's manifest cache persists across query runs within a session, so later queries in a run may understate cold-start latency. GKE node performance is subject to hypervisor scheduling, so observed latencies represent the platform as configured rather than an absolute ceiling on Trino's throughput.

7.11 Summary

This chapter presented five experiments evaluating TriBench across three deployment environments (local Docker, on-premises Kubernetes, and GKE), a cross-environment portability study, and a framework overhead measurement. The four reproducibility dimensions identified in Section 2.4 are each evidenced in Section 7.9. A cross-environment portability experiment confirmed that identical experiment definitions produced valid results on all three platforms with only a host configuration file change. Mean monitoring overhead remained below the 5% target across all environments. All Must-Have and Should-Have requirements were met; Appendix C provides a full traceability table.

8 Conclusion

8.1 Summary

TriBench is a benchmarking framework for Apache Trino data lakehouses. It gives researchers a single tool to define, run, and reproduce SQL benchmark experiments across different deployment environments, from a laptop to a cloud cluster, without manually configuring the underlying stack.

The framework was evaluated across three environments: a local machine, on premise kubernetes cluster, and Google Cloud. In all three cases the same experiment definition ran without modification, confirming that switching environments requires only a configuration change. Monitoring overhead stayed below 5% in every environment, and a seven-hour unattended suite on Google Cloud completed without intervention, showing the framework is practical at industry scale.

8.2 Reflection

This project was a substantial learning experience in areas that are difficult to acquire outside of a working system. Prior familiarity with Kubernetes, Apache Iceberg, and cloud-native deployment was limited before this project. Working through the full data lakehouse stack, configuring Hive Metastore, MinIO, and Trino as coordinated services, and understanding how they interact under analytical load, built practical knowledge in distributed systems and data engineering that formal coursework alone would not have provided.

Deploying across three different environments was the most genuinely exciting part of the project. There was real uncertainty about whether the same experiment definition would work unchanged on a university kubernetes cluster accessed via an SSH tunnel and a managed cloud cluster on GKE, two very different infrastructure models. Getting both to work through TriBench's profile system, without modifying a single experiment file, was a satisfying validation of the design and reinforced existing knowledge in cloud computing in a practical context.

The most underestimated challenge was integration rather than implementation. Each component in isolation was straightforward to build and test, but making the full stack behave reliably as a unit, particularly the dependency ordering of Kubernetes service startup and the port-forward tunnel lifecycle, consumed a disproportionate share of late-stage debugging time. Running at SF 100 also reinforced that some design gaps only become apparent at scale: a system that feels complete at SF 1 can reveal missing operational affordances once committed to a multi-hour run. Similarly, while component versions are configurable through the HOCON system (Section 5.2), no experiments were conducted to compare performance across Trino releases or Iceberg versions; a version sweep would have been a natural use of the framework and is an omission in hindsight.

Reading related work before committing to a design proved more valuable than anticipated. Studying PEEL, LST-Bench, and Benchto early shaped the requirements and prevented design decisions that would have duplicated existing work or missed known pitfalls. Writing this dissertation was the most substantial piece of academic writing undertaken so far, and working

through how to present a systems contribution, motivating the problem, situating it in related work, and evaluating it systematically, improved technical writing skills that will carry forward.

8.3 Future Work

Several directions would meaningfully extend TriBench.The current monitoring backend relies on psutil for host metrics and the Kubernetes Metrics Server for per-pod data; adding a Prometheus (The Linux Foundation 2026) metrics endpoint and a Grafana (Grafana Labs 2026) dashboard would allow real-time visibility during long experiment runs without post-hoc log inspection. The framework could also be extended to support Trino configuration as an experiment variable, for example automatically varying the JVM heap size or the number of worker nodes across an experiment sweep and aggregating results.

On the reproducibility side, the bundle archive format is currently a plain tar.gz; adopting a content-addressed format and a bundle registry would allow researchers to publish and retrieve reproducible benchmarks by hash, analogous to how container images are shared via a registry. The analysis layer currently provides per-run statistics; adding automatic regression tracking across successive runs would make TriBench useful as a continuous benchmarking tool in a CI pipeline, flagging query latency regressions introduced by configuration or schema changes.

A CLI Command Reference

This appendix provides a complete reference for all TriBench CLI command groups, their sub-commands, and arguments. The framework is operated entirely through this interface.

Command	Description
bundle – Bundle management commands	
`archive`	Archive the bundle's results and logs
`clear`	Clear the active bundle state
`create`	Scaffold a new bundle directory
`info`	Show bundle metadata and resolved paths
`list`	List all experiment YAML files in the bundle
`set`	Set a bundle as active persistently
`show`	Show the currently active bundle
`validate`	Check that the bundle has all required elements
sys – System lifecycle management commands	
`cluster`	Manage the Kind Kubernetes cluster
`logs`	Show system component logs
`port-forward`	Manage Kubernetes port forwarding tunnels
`setup`	Set up the lakehouse system components
`start`	Start the system and poll for readiness
`status`	Check current system status
`stop`	Stop the system gracefully
`teardown`	Tear down the system (destructive operation)
exp – Experiment execution commands	
`cancel`	Cancel a currently running experiment
`config`	Show the configuration for a specific experiment
`list`	List available experiments
`run`	Execute an experiment end-to-end
`status`	Check the status of a running experiment

Command	Description
res – Result viewing and basic analysis commands	
`analyze`	Analyze experiment results (statistics, performance, scalability, etc.)
`archive`	Archive old experiment results
`delete`	Delete an experiment result by ID
`export`	Export experiment results to CSV, JSON, or Parquet
`list`	List all recorded experiment results
`monitoring`	Show hardware monitoring telemetry for an experiment
`queries`	Show detailed per-query execution data
`reset-db`	Reset the SQLite database by deleting all runs
`show`	Show high-level summary of an experiment result
`suite-summary`	Show an aggregated summary across a suite of runs
data – Dataset management commands	
`generate`	Generate a benchmark dataset
`info`	Show detailed dataset information
`list`	List available datasets
`load`	Load a dataset into the deployed system
config – Configuration management commands	
`defaults`	Show framework defaults
`profile`	Manage active configuration profile
`show`	Show the current merged configuration
`trace`	Trace where a specific configuration value originated
`validate`	Validate the complete configuration for errors
suite – Experiment suite execution commands	
`list`	List available experiment suites
`run`	Execute all experiments in a designated suite
`show`	Show detailed information about a suite

Table A.1: *Complete reference of TriBench CLI commands, split across two tables.*

B Getting Started Guide

This guide covers installing TriBench from source, creating a bundle, and running a TPC-H experiment on a local Docker stack. For bare-metal Kubernetes deployment on the University of Glasgow GPG cluster, follow `docs/GPG/GPG_DEPLOYMENT.md` (single node) or `docs/GPG/GPG_MULTINODE_DEPLOYMENT.md` (multi-node) in the repository. For GKE, follow `docs/GKE/GCP_DEPLOYMENT.md`.

Prerequisites

- Anaconda or Miniconda (provides Python 3.11 and the `conda` environment manager)
- Docker Desktop (macOS / Windows) or Docker Engine (Linux)
- MinIO Client (`mc`) – required for object storage operations
- Git
- `kubectl` – Kubernetes backend only
- `gcloud` CLI – GKE only

Step 1 – Clone and install

```
git clone https://github.com/AdamLinYuan/tribench-framework
cd tribench-framework
conda env create -f environment.yml
conda activate tribench
pip install -e .
```

Verify the installation:

```
tribench --version
```

Step 2 – Create a bundle

A bundle is a self-contained directory that holds all experiment artefacts. Create one and set it as active:

```
tribench bundle create my-study
tribench bundle set my-study
```

Step 3 – Select the Docker profile

TriBench ships with a `docker` host configuration profile targeting a local single-node stack. Activate it:

```
tribench config profile docker
```

Step 4 – Start the lakehouse stack

```
tribench sys setup all
tribench sys start all
```

This pulls and starts five containers (Trino coordinator, one worker, Hive Metastore, MinIO, and PostgreSQL) and waits until all pass their health checks. The stack is typically ready in under 90 seconds.

Step 5 – Generate and load a dataset

TPC-H data is generated directly inside Trino and requires no local generation step. TPC-DS and custom datasets must be generated first. To load TPC-H at scale factor 1 (approximately 1 GB):

```
tribench data load tpch-sf1 --schema tpch_sf1
```

For TPC-DS, generate the data first and then load:

```
tribench data generate tpcds-sf10 --format parquet
tribench data load tpcds-sf10 --schema tpcds_sf10
```

For custom datasets, place one or more Parquet files inside a named folder under `datasets/` (either inside the active bundle or the framework root directory), then load by folder name:

```
tribench data load my_dataset --schema my_schema
```

Step 6 – Run an experiment

```
tribench exp run experiments/tpch-docker.yaml
```

Progress is printed to the terminal. Results are written automatically to the bundle's SQLite database at `my-study/results/tribench.db`.

Step 7 – Inspect and export results

```
tribench res list # list all recorded runs
tribench res show tpch-docker # per-query summary
tribench res analyze statistics tpch-docker
tribench res export tpch-docker --format csv
```

Step 8 – Stop the stack

```
tribench sys stop all
```

To fully remove all containers and volumes:

```
tribench sys teardown all
```

Reproducing the Evaluation Bundles

All bundles used in the dissertation evaluation (`docker`, `gpg`, `gcp-1w`, `gcp-2w`, `gcp-4w`) are included in the `bundles/` directory. The full datasets were removed from the repository due to size constraints. Small reference datasets (TPC-H SF 0.01, TPC-DS SF 0.01, and the custom e-commerce dataset) are included in `datasets/` and can be used to verify the framework without generating data.

To reproduce an evaluation bundle, activate it and generate the datasets referenced by its experiments:

```
tribench bundle set docker # or gpg, gcp-4w, etc.
tribench data generate tpch-sf1 --schema tpch_sf1
tribench data generate tpcds-sf10 --schema tpcds_sf10
```

The experiment YAML files inside each bundle list the exact datasets and queries used. Run any experiment with `tribench exp run` or the full suite with `tribench suite run` once the datasets are loaded.

Kubernetes deployments

To run TriBench on a Kubernetes cluster, switch the active profile to one that sets `backend = "kubernetes"` and points to the appropriate `kubectl` context. The CLI commands for experiment execution and result analysis are identical; only the host configuration file differs.

- **GPG single-node**: follow `docs/GPG/GPG_DEPLOYMENT.md`
- **GPG multi-node**: follow `docs/GPG/GPG_MULTINODE_DEPLOYMENT.md`
- **GKE**: follow `docs/GKE/GCP_DEPLOYMENT.md`

C Requirements Justification

Table C.1: *Requirements traceability. ✓ = met, N/A = deferred (Won't Have).*

Req.	Met	Evidence
M1	✓	Same YAML definitions ran unchanged on Docker, GPG, and GKE (§7.4, 7.5, 7.6).
M2	✓	TPC-H, TPC-DS, and a custom Parquet dataset all loaded and queried successfully (§7.4–7.6).
M3	✓	Warmup runs, run counts, and per-query timeouts set via YAML in every experiment without code changes.
M4	✓	CPU and memory sampled continuously throughout every run (§7.4.3).
M5	✓	Repeated invocations appended new records without overwriting prior results (§7.4.4, Appendix D).
M6	✓	Results exported to CSV after every evaluation run (§7.4.4).
M7	✓	All five analysis subcommands produced output against the Docker TPC-H dataset (Appendix D).
M8	✓	Docker and Kubernetes stacks started and torn down automatically in every session (§7.4–7.6).
M9	✓	Switching environment required only a host configuration file change (§7.7).
M10	✓	Bundle directories archived all artefacts and reproduced cleanly from a fresh checkout.
M11	✓	`tribench bundle create` and `activate` used to scaffold and set context for every evaluation bundle.
M12	✓	TPC-H SF 100 replicated across five runs on GKE with low inter-run variance (§7.6.4).
M13	✓	Identical YAML produced complete results on macOS, Linux (GPG), and Linux (GKE) (§7.7).
M14	✓	Structured timestamped logs emitted throughout all sessions at configurable verbosity.
S1	✓	Seven-hour unattended suite ran all four GKE experiments under a shared lifecycle (§7.6.3).
S2	✓	Per-pod CPU and memory collected via the Kubernetes Metrics Server; trace shown in §7.6.3.
S3	✓	Mean monitoring overhead below 5% across all three environments (§7.8).
C1	✓	Docker and Kubernetes each implemented as an independent subclass; no orchestration code changed to add either (§6.8).
W1	N/A	Alternative SQL engine support deferred; out of scope.

D Screenshots

The following screenshots show example terminal output from `tribench res show` and each of the five `tribench res analyze` subcommands. Each command is run against a real experiment recorded during evaluation.

```
❯ tribench res show 4 --runs

================================================================================
Experiment: tpch-all-2 (ID: 4)
================================================================================
Type:       trino_query
Dataset:    tpch-sf1-iceberg
Tags:       tpch, sf1, docker, baseline, all-queries
Created:    2026-03-11 19:03:25

Total Runs: 3

--------------------------------------------------------------------------------------------------
Run ID   Run    Status       Duration(s)     Queries    Success    Failed     Monitoring
--------------------------------------------------------------------------------------------------
16       3      completed    7.10            22         22         0          350 metrics
15       2      completed    7.17            22         22         0          265 metrics
14       1      completed    7.72            22         22         0          715 metrics
--------------------------------------------------------------------------------------------------
================================================================================
```

Figure D.1: *Output of* `tribench res show <experiment_id> –runs`*: accumulated run records in the bundle SQLite database after two successive experiment invocations (**M5**).*

```
❯ tribench res analyze statistics 10

Statistical Analysis for Experiment 10: tpch-all-sf10
Total Runs: 3

Query: q01
  Sample Size: 3
  Mean:        2.911s
  Median:      2.925s
  Std Dev:     0.072s
  Min:         2.834s
  Max:         2.975s
  P50:         2.925s
  P95:         2.970s
  P99:         2.974s
  95% CI:      [2.733s, 3.089s]
  Outliers:    0 detected

Query: q02
  Sample Size: 3
  Mean:        2.952s
  Median:      2.846s
  Std Dev:     0.193s
  Min:         2.834s
  Max:         3.175s
  P50:         2.846s
  P95:         3.142s
  P99:         3.168s
  95% CI:      [2.472s, 3.432s]
  Outliers:    0 detected

Query: q03
  Sample Size: 3
  Mean:        3.730s
  Median:      3.597s
  Std Dev:     0.245s
  Min:         3.580s
  Max:         4.013s
  P50:         3.597s
  P95:         3.971s
  P99:         4.005s
  95% CI:      [3.121s, 4.339s]
  Outliers:    0 detected
```

Figure D.2: *Output of* `tribench res analyze statistics`*: per-query mean, median, standard deviation, and P95 latency across measured runs, with IQR-based outlier flags.*

```
❯ tribench res analyze performance 11

Performance Analysis for Experiment 11
Total Runs: 3
Total Queries Executed: 65

Overall Execution Time Statistics:
  count: 65
  mean: 418.313s
  median: 413.701s
  stdev: 243.580s
  variance: 59331.445s
  cv: 58.229s
  min: 45.543s
  max: 921.214s
  range: 875.671s
  p25: 266.564s
  p50: 413.701s
  p75: 569.898s
  p90: 774.732s
  p95: 799.681s
  p99: 888.713s
  iqr: 303.334s
```

Figure D.3: *Output of* `tribench res analyze performance`*: overall throughput in queries per second and per-query min/mean/max latency, run against the TPC-H SF 100 dataset on GKE.*

```
❯ tribench res analyze compare 12 11

================================================================================
COMPREHENSIVE EXPERIMENT COMPARISON
================================================================================
Baseline: overhead-off5 (ID: 12)
Current:  overhead-on4 (ID: 11)
Significance Level: 0.05

OVERALL PERFORMANCE STATISTICS
================================================================================
Metric           Baseline         Current          Difference       % Change
--------------------------------------------------------------------------------
total_runtime    75.88s           78.28s           +2.40s           +3.17%
total_queries    90               90               +0
success_rate     100.0%           100.0%           +0.0pp
--------------------------------------------------------------------------------
count            90               90               +0               +0.00%
mean             0.842s           0.868s           +0.026s          +3.11%
median           0.404s           0.415s           +0.010s          +2.58%
stdev            0.841s           0.854s           +0.014s          +1.62%
min              0.098s           0.101s           +0.003s          +3.28%
max              2.312s           2.339s           +0.027s          +1.18%
p50              0.404s           0.415s           +0.010s          +2.58%
p90              2.154s           2.168s           +0.014s          +0.66%
p95              2.230s           2.213s           -0.017s          -0.76%
p99              2.269s           2.329s           +0.061s          +2.68%

PER-QUERY PERFORMANCE COMPARISON
================================================================================
Query           Baseline     Current      Diff         % Change     Verdict
--------------------------------------------------------------------------------
q01                  0.417s      0.445s     +0.028s      +6.72% No Change
q06                  0.114s      0.117s     +0.002s      +2.14% No Change
q18                  1.994s      2.042s     +0.048s      +2.41% No Change

SUMMARY
================================================================================
Queries Compared:      3
Improvements:          0 queries
Regressions:           0 queries
No Significant Change: 3 queries
Overall Change:        +3.11% (slower)

================================================================================
```

Figure D.4: *Output of* `tribench res analyze compare`*: head-to-head comparison of the instrumented and uninstrumented overhead runs on the local Docker stack, showing per-query latency difference and t-test significance.*

```
❯ tribench res analyze regression 2 3

Performance Regression Detection
Baseline: Experiment 2
Current:  Experiment 3
Threshold: ≥5.0% slowdown
Significance Level: 0.05

No regressions detected (threshold: 5.0%)

1 Performance Improvement(s):

  + q18
      Faster by: 6.16%

Summary:
  Total Queries:  3
  Regressions:    0
  Improvements:   1
  No Change:      2
```

Figure D.5: *Output of* `tribench res analyze regression`*: queries flagged by severity band (minor, moderate, major, critical) with dual-gate threshold and t-test confirmation, comparing the instrumented and uninstrumented overhead runs on GKE with four workers.*

```
❯ tribench res analyze scalability 12 4 --baseline-workers 2 --scaled-workers 4

Scalability Analysis
Baseline: Experiment 12 - 2 worker(s)
Scaled:   Experiment 4 - 4 worker(s)

Overall Metrics:
  Average Speed-up: 0.96x
  Median Speed-up:  0.96x
  Min Speed-up:     0.88x
  Max Speed-up:     1.02x
  Average Efficiency: 48.1%
  Ideal Speed-up:     2.00x
  Assessment:         Poor

Per-Query Speed-up:
  [slowdown] q01:
      Baseline: 2.068s
      Scaled:   2.164s
      Speed-up: 0.96x
      Efficiency: 47.8%
  [speedup] q06:
      Baseline: 1.905s
      Scaled:   1.899s
      Speed-up: 1.00x
      Efficiency: 50.2%
  [slowdown] q09:
      Baseline: 2.535s
      Scaled:   2.878s
      Speed-up: 0.88x
      Efficiency: 44.0%
  [slowdown] q18:
      Baseline: 2.488s
      Scaled:   2.613s
      Speed-up: 0.95x
      Efficiency: 47.6%
  [speedup] q21:
      Baseline: 3.205s
      Scaled:   3.147s
      Speed-up: 1.02x
      Efficiency: 50.9%
```

Figure D.6: *Output of* `tribench res analyze scalability`*: per-query speed-up and parallel efficiency comparing the two-worker and four-worker GKE portability runs on TPC-H SF 1.*

The following figure details the line coverage per module across the tested components of TriBench. The test suite comprises 267 unit tests written using `pytest`, achieving an overall coverage of 85%.

```
267 passed, 4 warnings in 1.02s
Name                                                    Stmts   Miss  Cover
-----------------------------------------------------------------------------
lib/tribench/analysis/statistical.py                       98     14    86%
lib/tribench/bundle/manifest.py                           170     22    87%
lib/tribench/cli/base.py                                  123      5    96%
lib/tribench/config/connection.py                         143     20    86%
lib/tribench/core/experiment.py                           102     15    85%
lib/tribench/core/result.py                                27      0   100%
lib/tribench/defaults.py                                  140     16    89%
lib/tribench/experiments/query_executor.py                192     68    65%
69
lib/tribench/monitoring/base.py                           161     20    88%
lib/tribench/storage/connection.py                         99     31    69%
lib/tribench/storage/models.py                            123      0   100%
lib/tribench/storage/result/__init__.py                     6      0   100%
lib/tribench/storage/result/experiment_store.py            48      7    85%
lib/tribench/storage/result/metric_store.py                72      7    90%
lib/tribench/storage/result/query_store.py                 21      2    90%
lib/tribench/storage/result/result_storage.py              42      2    95%
lib/tribench/storage/result/run_store.py                   59     10    83%
-----------------------------------------------------------------------------
TOTAL                                                    1626    239    85%
```

Figure D.7: *Test coverage report from* `pytest --cov=. --cov-report=term`

```yaml
name: "custom-gpg"
description: "Custom e-commerce domain queries — GPG bare-metal K8s (2 workers)"

system: "trino"

connection:
  host: "localhost"
  port: 8080
  user: "tribench"
  catalog: "iceberg"
  schema: "ecommerce"

runs: 3
warmup_runs: 1
timeout_seconds: 300
max_retries: 2

monitoring:
  enabled: true
  interval_seconds: 5.0
  kubernetes:
    enabled: true
    context: "gpg-multinode"
    namespace: "tribench"
    label_selector: "app=trino"
    pod_name_pattern: "trino-.*"

queries:
  - name: "monthly_revenue_by_category"
    description: "Monthly revenue trend broken down by product category"
    sql: |
      SELECT
        date_trunc('month', CAST(o.order_date AS DATE)) AS month,
        p.category,
        COUNT(DISTINCT o.order_id)                      AS order_count,
        SUM(oi.quantity * oi.unit_price)                AS revenue
      FROM orders o
      JOIN order_items oi ON o.order_id = oi.order_id
      JOIN products p     ON oi.product_id = p.product_id
      WHERE o.status = 'delivered'
      GROUP BY 1, 2
      ORDER BY month, revenue DESC

  - name: "customer_segment_summary"
    description: "Order volume and spend statistics per customer segment and country"
    sql: |
      SELECT
        c.segment,
        c.country,
        COUNT(DISTINCT c.customer_id)  AS customer_count,
        COUNT(DISTINCT o.order_id)     AS total_orders,
        SUM(o.total_amount)            AS total_revenue,
        AVG(o.total_amount)            AS avg_order_value
      FROM customers c
      JOIN orders o ON c.customer_id = o.customer_id
      GROUP BY c.segment, c.country
      ORDER BY total_revenue DESC

validation:
  min_success_rate: 1.0
```

Figure D.8: *custom e-commerce experiment definition from Section 7.5.3, illustrating inline SQL query blocks defined directly in the experiment YAML without separate* `.sql` *files.*

The following figure shows the Iceberg snapshot provenance record captured by `tribench data info` after loading, showing the snapshot identifier and commit timestamp per table.

```yaml
tpch-sf1-iceberg:
  name: tpch-sf1-iceberg
  benchmark_type: tpch
  type: static
  format: iceberg
  scale_factor: 1.0
  size_bytes: null
  location: iceberg.tpch_sf1
  tables:
  - nation
  - region
  - customer
  - supplier
  - part
  - partsupp
  - orders
  - lineitem
  row_counts:
    nation: 25
    region: 5
    customer: 150000
    supplier: 10000
    part: 200000
    partsupp: 800000
    orders: 1500000
    lineitem: 6001215
  checksums: {}
  properties:
    source_dataset: tpch-sf1
    partitioned: false
    storage_location: default
  created_at: '2025-12-19T12:33:21.421558'
  generator: iceberg_loader
  iceberg_catalog: iceberg
  iceberg_schema: tpch_sf1
  snapshot_ids:
    nation: 8283669988127154758
    region: 4813755681681115579
    customer: 6611341530930247962
    supplier: 2442304681037913495
```

E Source Code Directory Structure

The following listing shows the top-level layout of the TriBench source repository. Generated artefacts (compiled bytecode, virtual environments, coverage reports, and backup files) are excluded. The `lib/tribench` package is the primary deliverable; all other top-level directories are supporting resources.

```
tribench-framework/
|-- config/
| |-- reference.conf # framework defaults (layer 1)
| |-- hosts/ # named deployment profiles (layer 2)
| `-- templates/ # Jinja2 configuration templates
|-- experiments/ # experiment YAML definitions
| |-- suites/ # suite YAML files
| `-- templates/ # scaffolded experiment templates
|-- lib/
| `-- tribench/
| |-- analysis/ # statistical analysis commands
| | |-- comparison.py
| | |-- performance.py
| | |-- regression.py
| | |-- scalability.py
| | `-- statistical.py
| |-- bundle/
| | `-- manifest.py # bundle path resolution
| |-- cli/ # Click command groups
| | |-- base.py
| | |-- bundle_commands.py
| | |-- config_commands.py
| | |-- experiment_commands.py
| | |-- data/ # data subcommands
| | |-- result/ # result subcommands
| | |-- suite/ # suite subcommands
| | `-- system/ # sys subcommands
| |-- core/ # shared abstractions
| | |-- experiment.py
| | |-- experiment_registry.py
| | |-- experiment_suite.py
| | |-- result.py
| | `-- system.py # system interface definition
| |-- data/ # dataset management
| | |-- dataset/ # registry, schema, loaders
| | `-- iceberg/ # built-in and universal loaders
| |-- experiments/ # experiment execution engine
| | |-- query_executor.py
| | |-- result_collector.py
| | `-- trino/ # Trino-specific execution
| |-- monitoring/ # monitoring subsystem
| | |-- base.py
| | |-- resource_monitor.py
| | |-- kubernetes_monitor.py
| | `-- trino/ # Trino query statistics collector
| |-- storage/ # result persistence
| | |-- models.py
| | |-- connection.py
| | `-- result/ # per-entity store modules
```

```
| |-- systems/ # deployment backends
| | |-- trino/ # Trino service lifecycle
| | |-- kubernetes/ # Kubernetes backend
| | |-- hive_metastore.py
| | |-- minio.py
| | `-- postgresql.py
| `-- utils/
| `-- config/ # HOCON loading and merging
|-- queries/
| |-- tpch/ # 22 TPC-H SQL files
| `-- tpcds/ # 99 TPC-DS SQL files
|-- systems/ # service configuration assets
| |-- trino-434/
| |-- hive-metastore-4.0.0/
| |-- minio/
| |-- postgresql-15/
| `-- kubernetes/ # Kubernetes manifest templates
|-- tests/ # pytest test suite (267 tests)
| |-- conftest.py
| |-- test_analysis.py
| |-- test_bundle.py
| |-- test_cli.py
| |-- test_config.py
| |-- test_core.py
| |-- test_execution.py
| |-- test_kubernetes_monitor.py
| |-- test_monitoring.py
| `-- test_storage_models.py
|-- utils/
| `-- plot_eval.py # evaluation figure generation
|-- environment.yml # conda environment definition
|-- setup.py # package entry point
`-- pytest.ini
```

Bibliography

Abadi, D. (2012), 'The design and implementation of modern column-oriented database systems', *Foundations and Trends in Databases* **5**, 197–280.

Abadi, D., Madden, S. and Hachem, N. (2008), Column-stores vs. row-stores: How different are they really?, pp. 967–980.

Amdahl, G. (2000), *Validity of the Single-Processor Approach to Achieving Large Scale Computing Capabilities*, Vol. 12, pp. 79–81.

Apache Software Foundation (2026*a*), 'Apache hudi: Large-scale data processing on cloud-native storage', `https://hudi.apache.org`.

Apache Software Foundation (2026*b*), 'Apache Iceberg: An open table format for huge analytic datasets', `https://iceberg.apache.org`.

Apache Software Foundation (2026*c*), 'Apache parquet', `https://parquet.apache.org`.

Armbrust, M., Das, T., Sun, L., Yavuz, B., Zhu, S., Murthy, M., Torres, J., Hovell, H., Ionescu, A., Łuszczak, A., Świtakowski, M., Szafrański, M., Li, X., Ueshin, T., Mokhtar, M., Boncz, P., Ghodsi, A., Paranjpye, S., Senster, P. and Zaharia, M. (2020), 'Delta lake: high-performance acid table storage over cloud object stores', *Proceedings of the VLDB Endowment* **13**, 3411–3424.

Bayer, M. (2026), 'SQLAlchemy: The Python SQL toolkit and object relational mapper', `https://www.sqlalchemy.org`.

Boden, C., Alexandrov, A., Kunft, A., Rabl, T. and Markl, V. (2015), 'PEEL: A system for benchmarking distributed data processing systems', `https://github.com/stratosphere/peel`. Accessed: March 2026.

Boden, C., Alexandrov, A., Kunft, A., Rabl, T. and Markl, V. (2017), Peel: A framework for benchmarking distributed systems and algorithms, pp. 9–24.

Camacho-Rodríguez, J., Agrawal, A., Gruenheid, A., Gosalia, A., Petculescu, C., Aguilar-Saborit, J., Floratou, A., Curino, C. and Ramakrishnan, R. (2024), 'Lst-bench: Benchmarking log-structured tables in the cloud', *Proceedings of the ACM on Management of Data* **2**, 1–26.

Carbone, P., Katsifodimos, A., Kth, Sweden, S., Ewen, S., Markl, V., Haridi, S. and Tzoumas, K. (2015), 'Apache flink™: Stream and batch processing in a single engine', *IEEE Data Engineering Bulletin* **38**.

Clegg, D. and Barker, R. (1994), *CASE method fast-track - a RAD approach.*

Collberg, C. S. and Proebsting, T. A. (2016), 'Repeatability in computer systems research', *Communications of the ACM* **59**(3), 62–69.
URL: *http://doi.acm.org/10.1145/2812803*

Daniel Palma (2022), 'High-performance open-source data lakehouse at home', https://www.arecadata.com/high-performance-open-source-data-lakehouse-at-home/. Accessed: March 2026.

Fuller, M., Baier, M. and Traverso, M. (2021), *Trino: The Definitive Guide*, O'Reilly Media, Sebastopol, CA.

Georges, A., Buytaert, D. and Eeckhout, L. (2007), Statistically rigorous java performance evaluation, *in* R. P. Gabriel, D. F. Bacon, C. V. Lopes and G. L. S. Jr., eds, 'Proceedings of the 22nd Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA 2007, October 21-25, 2007, Montreal, Quebec, Canada', ACM, pp. 57–76.
URL: *http://doi.acm.org/10.1145/1297027.1297033*

Grafana Labs (2026), 'Grafana: The open observability platform', https://grafana.com.

Gray, J., ed. (1993), *The Benchmark Handbook for Database and Transaction Systems (2nd Edition)*, Morgan Kaufmann.

Hipp, R. D. (2026), 'SQLite: A self-contained, serverless, zero-configuration sql database engine', https://www.sqlite.org.

HPCwire (2025), 'Snowflake bolsters support for Apache Iceberg tables', https://www.hpcwire.com/bigdatawire/2025/04/08/snowflake-bolsters-support-for-apache-iceberg-tables/. Accessed: March 2026.

Huppler, K. (2009), The art of building a good benchmark, *in* R. O. Nambiar and M. Poess, eds, 'Performance Evaluation and Benchmarking, First TPC Technology Conference, TPCTC 2009, Lyon, France, August 24-28, 2009, Revised Selected Papers', Vol. 5895 of *Lecture Notes in Computer Science*, Springer, pp. 18–30.
URL: *http://dx.doi.org/10.1007/978-3-642-10424-4*$_3$

Icon Icons (2026), 'PostgreSQL plain wordmark logo', https://icon-icons.com/icon/postgresql-plain-wordmark-logo/146390.

Jain, R. (1990), 'The art of computer systems performance analysis', *SIGMETRICS Performance Evaluation Review* **18**(3), 21–22.

Kimball, R. and Ross, M. (2013), *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*, 3rd edn, Wiley, Indianapolis, IN.

Kubernetes SIGs (2026), 'KinD: Kubernetes in docker', https://kind.sigs.k8s.io.

Lightbend (2026), 'Hocon (human-optimized config object notation)', https://github.com/lightbend/config/blob/master/HOCON.md.

McKinney, W. (2010), Data structures for statistical computing in python, *in* S. van der Walt and J. Millman, eds, 'Proceedings of the 9th Python in Science Conference 2010 (SciPy 2010), Austin, Texas, June 28 - July 3, 2010', scipy.org, pp. 56–61.
URL: *https://doi.org/10.25080/Majora-92bf1922-00a*

Microsoft (2025), 'Review and configure code coverage results in azure pipelines', https://learn.microsoft.com/en-us/azure/devops/pipelines/test/review-code-coverage-results. Accessed: March 2026.

MinIO Inc. (2026), 'MinIO: High-performance object storage', https://min.io.

Nambiar, R. O. and Poess, M. (2006), The making of tpc-ds, *in* U. Dayal, K.-Y. Whang, D. B. Lomet, G. Alonso, G. M. Lohman, M. L. Kersten, S. K. Cha and Y.-K. Kim, eds, 'Proceedings of the 32nd International Conference on Very Large Data Bases, Seoul, Korea, September 12-15, 2006', ACM, pp. 1049–1058.
URL: *http://www.vldb.org/conf/2006/p1049-othayoth.pdf*

Open Policy Agent (2026), 'MinIO — OPA ecosystem', `https://www.openpolicyagent.org/ecosystem/entry/minio`.

Pallets (2026*a*), 'Click: Python command line interface framework', `https://click.palletsprojects.com/`.

Pallets (2026*b*), 'Jinja: A fast and designer-friendly templating language', `https://jinja.palletsprojects.com`.

Poess, M., Rabl, T. and Jacobsen, H.-A. (2017), Analysis of tpc-ds: the first standard benchmark for sql-based big data systems, *in* 'Proceedings of the 2017 Symposium on Cloud Computing, SoCC 2017, Santa Clara, CA, USA, September 24 - 27, 2017', ACM, pp. 573–585.
URL: *http://doi.acm.org/10.1145/3127479.3128603*

PrestoDB (2026), 'benchto: Framework for running macro benchmarks in a clustered environment', `https://github.com/prestodb/benchto`. Accessed: March 2026.

pytest Contributors (2026), 'pytest: A mature full-featured Python testing framework', `https://pytest.org`.

Raasveldt, M. and Mühleisen, H. (2019), Duckdb: an embeddable analytical database, *in* P. A. Boncz, S. Manegold, A. Ailamaki, A. Deshpande and T. Kraska, eds, 'Proceedings of the 2019 International Conference on Management of Data, SIGMOD Conference 2019, Amsterdam, The Netherlands, June 30 - July 5, 2019', ACM, pp. 1981–1984.
URL: *https://doi.org/10.1145/3299869.3320212*

Rodola, G. (2026), 'psutil: Cross-platform library for process and system monitoring in Python', `https://github.com/giampaolo/psutil`.

Sethi, R., Traverso, M., Sundstrom, D., Phillips, D., Xie, W., Sun, Y., Yegitbasi, N., Jin, H., Hwang, E., Shingte, N. and Berner, C. (2019), Presto: Sql on everything, *in* '35th IEEE International Conference on Data Engineering, ICDE 2019, Macao, China, April 8-11, 2019', IEEE, pp. 1802–1813.
URL: *https://doi.org/10.1109/ICDE.2019.00196*

Shvachko, K., Kuang, H., Radia, S. and Chansler, R. (2010), 'The hadoop distributed file system mass storage systems and technologies (msst) 2010', *IEEE 26th Symposium on* .

The Kubernetes Authors (2026), 'Kubernetes: Production-grade container orchestration'.
URL: *https://kubernetes.io*

The Linux Foundation (2026), 'Prometheus: Monitoring system and time series database', `https://prometheus.io`.

The PostgreSQL Global Development Group (2026), 'PostgreSQL: The world's most advanced open source relational database', `https://www.postgresql.org`.

Thusoo, A., Sarma, J. S., Jain, N., Shao, Z., Chakka, P., Anthony, S., Liu, H., Wyckoff, P. and Murthy, R. (2009), 'Hive - a warehousing solution over a map-reduce framework', *PVLDB* **2**(2), 1626–1629.
URL: *http://www.vldb.org/pvldb/2/vldb09-938.pdf*

Transaction Processing Performance Council (2026*a*), TPC-DS benchmark specification, revision 3.2.0, Technical report, Transaction Processing Performance Council.

Transaction Processing Performance Council (2026*b*), TPC-H benchmark specification, revision 3.0.1, Technical report, Transaction Processing Performance Council.

Trino Project (2021), 'What is trino and the hive connector', Trino Community Broadcast, `https://trino.io/episodes/29.html`. Accessed: March 2026.

Trino Project (2026), 'Trino foundation', `https://trino.io/foundation.html`.

Wikimedia Commons (2026), 'Apache Hive logo', `https://commons.wikimedia.org/wiki/File:Apache_Hive_logo.svg`.

Zaharia, M., 0002, A. G., Xin, R. and Armbrust, M. (2021), Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics, *in* '11th Conference on Innovative Data Systems Research, CIDR 2021, Virtual Event, January 11-15, 2021, Online Proceedings', www.cidrdb.org.
URL: *http://cidrdb.org/cidr2021/papers/cidr2021*$_p aper17.pdf$

Zaharia, M., Xin, R., Wendell, P., Das, T., Armbrust, M., Dave, A., Meng, X., Rosen, J., Venkataraman, S., Franklin, M., Ghodsi, A., Gonzalez, J., Shenker, S. and Stoica, I. (2016), 'Apache spark: A unified engine for big data processing', *Communications of the ACM* **59**, 56–65.